



**ΔΙΕΘΝΕΣ ΠΑΝΕΠΙΣΤΗΜΙΟ ΤΗΣ ΕΛΛΑΔΟΣ
ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ,
ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ**

**Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με
δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για
εφαρμογή σε προβλήματα κατηγοριοποίησης.**

**Πτυχιακή Εργασία του
Εμμανουήλ Μαραντίδης (3849)**

Επιβλέπων: Σ. Καζαρλής Επίκουρος Καθηγητής

ΣΕΡΡΕΣ, 2025

Υπεύθυνη Δήλωση: Βεβαιώνω ότι είμαι συγγραφέας αυτής της πτυχιακής εργασίας και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της, είναι πλήρως αναγνωρισμένη και αναφέρεται στην πτυχιακή εργασία. Επίσης έχω αναφέρει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών ή λέξεων, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επίσης, βεβαιώνω ότι αυτή η πτυχιακή εργασία προετοιμάστηκε από εμένα προσωπικά ειδικά για τις απαιτήσεις του προγράμματος σπουδών του Τμήματος Μηχανικών Πληροφορικής, Υπολογιστών και Τηλεπικοινωνιών του Διεθνούς Πανεπιστημίου της Ελλάδας.

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

Περιεχόμενα

1. Περίληψη	6
2. Περιγραφή του θέματος και των απαιτήσεών του.	8
2.1.1 Θέμα της Εργασίας.	8
2.1.2 Απαιτήσεις Υλοποίησης.	8
3. Σύντομη εισαγωγή στους Εξελικτικούς Αλγόριθμους, τους Γενετικούς Αλγόριθμους και τα Classifier Systems - αναφορά στη μηχανική μάθηση.	10
3.1 Εξελικτικοί Αλγόριθμοι (EA).....	10
3.2 Γενετικοί Αλγόριθμοι (ΓΑ)	11
3.3 Συστήματα Ταξινόμησης (Classifier Systems)	11
3.4 Σχέση με τη Μηχανική Μάθηση	12
4. Τι είναι και πώς λειτουργεί ένα Classifier System (CS)	13
4.1 Εισαγωγή	13
4.2 Αρχιτεκτονική ενός Classifier System	13
4.2.1 Μηχανισμός Αναγνώρισης (Matching)	13
4.2.2 Επιλογή Ενέργειας (Action Selection)	14
4.2.3 Μηχανισμός Ανταμοιβής (Reinforcement Learning)	14
4.2.4 Γενετικός Αλγόριθμος (Genetic Algorithm)	14
4.3 Λειτουργία ενός CS – Βήμα προς Βήμα	14
4.4 Είδη Classifier Systems	15
4.5 Εφαρμογές	16
4.6 Πλεονεκτήματα και Μειονεκτήματα	16
4.7 Συμπεράσματα	17
5. Περιγραφή του συστήματος ταξινόμησης (CS) και του προβλήματος που θα αντιμετωπίσει.....	17
5.1. Η αρχιτεκτονική του συστήματος ταξινόμησης (ClassifierSystem).....	18
5.1.1 Πληθυσμός κανόνων (Population of Rules).....	18
5.1.2 Αντιστοίχιση και ενεργοποίηση κανόνων (Rule Matching and Activation).....	18
5.1.3 Ενίσχυση μέσω του αλγορίθμου Bucket Brigade(Reinforcement via the Bucket Brigade Algorithm)	19

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

5.1.4 Γενετικός αλγόριθμος για την εξέλιξη (Genetic Algorithm for Evolution).....	19
5.2 Το πρόβλημα: Εκμάθηση του πολυπλέκτη (Multiplexer) 4 προς 1.....	20
5.3 Σκοπός και μαθησιακός στόχος.....	22
5.4 Λειτουργία του αναπτυγμένου CS	23
5.5 Σημασία της έρευνας.....	25
5.6 Αναμενόμενα αποτελέσματα.....	26
5.7 Ευρύτερο πλαίσιο και εφαρμογές.....	27
5.8 Συμπέρασμα.....	28
6 Τεχνική περιγραφή του κώδικα που αναπτύχθηκε.....	29
6.1 Επισκόπηση συστήματος	29
6.2 Αναπαράσταση κανόνων	30
6.3 Δεδομένα εκπαίδευσης και ορισμός του προβλήματος	31
6.4 Μηχανισμός αντιστοίχισης	32
6.5 Αξιολόγηση και ενίσχυση της καταλληλότητας (αλγόριθμος Bucket Brigade)	33
6.6 Εξελικτική μάθηση (γενετικός αλγόριθμος)	34
6.7 Ροή προσομοίωσης	35
6.8 Έξοδος και αναφορά	37
6.9 Παραμετροποίηση και έλεγχος χρήστη	38
6.10 Τεχνικά χαρακτηριστικά	39
7 Περιγραφή της ρύθμισης και των αποτελεσμάτων των εκτελεσθέντων κύκλων.....	40
7.1 Πειραματική διάταξη.	41
7.1.1 Πρόβλημα και δεδομένα εκπαίδευσης.	41
7.1.2 Βασική αλγοριθμική διαμόρφωση.	41
7.2 Αποτελέσματα για σύντομες εξελικτικές εκτελέσεις (2 000 γενιές). ...	44
7.3 Αποτελέσματα για μεσαίες εξελικτικές εκτελέσεις (6 000 γενιές). ...	46
7.4 Αποτελέσματα για μακρές εξελικτικές εκτελέσεις (14 000 γενιές).	48
7.5 Παγκόσμιες τάσεις και ποιοτικές παρατηρήσεις.	49
7.6 Περίληψη.	51
8 Περιγραφή της δεύτερης εφαρμογής των κανόνων που αναπτύχθηκαν από τον ΓΑ, για την εξαγωγή συμπερασμάτων.	52
8.1 Εισαγωγή: φόρτωση της εξελιγμένης βάσης κανόνων από την έξοδο ΓΑ.	53
8.2 Δημιουργία όλων των πιθανών εισόδων και εξόδων αναφοράς.	54
8.3 Εφαρμογή των κανόνων: ενεργοποίηση, αντιστοίχιση και επιλογή νικητή.	55
8.4 Λεπτομερής αναφορά ανά είσοδο.	57

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

8.5 Ανάλυση απόδοσης παγκόσμιων κανόνων.	57
8.6 Ο ρόλος αυτής της δεύτερης εφαρμογής για την εξαγωγή συμπερασμάτων. ...	59
9. Σχολιασμός των αποτελεσμάτων με 100% ακρίβεια.	60
9.1 Γενική επισκόπηση των αποτελεσμάτων σε όλες τις δοκιμές.	60
9.2 Επίδραση των ρυθμίσεων παραμέτρων στα αποτελέσματα.	61
9.2.1 Ποσοστό μετάλλαξης (0,01).	61
9.2.2 Μέγεθος πληθυσμού (45) και μέγιστος αριθμός κανόνων (60).	62
9.2.3 Ποσοστό προσφοράς (0,05) και σύστημα ανταμοιβής.	62
9.2.4 Ελιτισμός (Elitism) (0,25).	63
9.2.5 Παράγοντας αρνητικής ανταμοιβής (Negative Reward Factor) (1,25).	63
9.3 Λεπτομερής σχολιασμός για τις εκτελέσεις με 100% ακρίβεια.	64
9.3.1 Τα τέλεια σύνολα κανόνων περιέχουν πάντα καθαρά σχήματα που γενικεύονται σωστά.	64
9.3.2 Κυριαρχία των κανόνων που καθορίζονται από τον επιλογέα.	65
9.3.3 Απουσία υπερβολικά γενικευμένων ή υπογενικευμένων κανόνων. ...	66
9.4 Γιατί ορισμένες εκτελέσεις φτάνουν το 100% και άλλες όχι.	66
9.4.1 Η έγκαιρη ανακάλυψη κανόνων έχει σημασία.	67
9.4.2 Η γενετική ποικιλομορφία παίζει σημαντικό ρόλο.	67
9.4.3 Η ισορροπία μεταξύ εξερεύνησης και εκμετάλλευσης είναι κρίσιμη.	67
9.5 Συνολική ερμηνεία.	68
10. Συμπεράσματα και οφέλη από την παρούσα εργασία.	69
11. Προτάσεις για μελλοντική επέκταση.	71
Βιβλιογραφία-Bibliography	76
Παράρτημα Α	80
Παράρτημα Β	88

Περίληψη

Τίτλος : Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

Ο σκοπός της εργασίας αυτής είναι η ανάπτυξη ενός παραδείγματος Classifier System με την δυνατότητα να εξελίσσει με γενετικό τρόπο σετ από κανόνες (if then rules) οι οποίοι προσομοιάζουν μια επιθυμητή συμπεριφορά εισόδου – εξόδου. Θα αναπτυχθεί ένα μοντέλο CS που θα έχει σταθερό πλήθος κανόνων, με «δυνάμεις κανόνων» και αλγόριθμο απονομής βαθμών (Bucket Brigade Algorithm), το οποίο θα εφαρμοστεί σε απλό πρόβλημα κατηγοριοποίησης (Classification). Το σετ κανόνων του CS θα εξελίσσεται γενετικά με τη βοήθεια Γενετικού Αλγόριθμου, ο οποίος θα χρησιμοποιεί :

1. Πληθυσμό λύσεων (κανόνων)
2. Μέθοδο επιλογής γονέων (Τροχός της Ρουλέτας)
3. Τελεστή Ανασυνδυασμού πολλαπλών σημείων (Multi-point Crossover)
4. Τελεστή Μετάλλαξης (Mutation)
5. Ελιτισμό
6. Μεταβλητά κριτήρια τερματισμού
7. Παραμέτρους καθοριζόμενες από τον χρήστη

Στην εργασία αυτή θα γίνει και μελέτη της αποτελεσματικότητας του CS στα προβλήματα εφαρμογής, για διάφορες τιμές των παραμέτρων.

Abstract

Title : Implementation of a Classifier System with a fixed number of rules, capable of chain rule activation (reasoning) for application to clustering problems.

The purpose of this work is to develop an example Classifier System with the ability to genetically evolve sets of rules (if then rules) that simulate a desired input-output behavior. A CS model will be developed that has a fixed set of rules, with "rule forces" and a Bucket Brigade Algorithm, which will be applied to a simple classification problem (Classification) The CS rule set will be genetically evolved using a Genetic Algorithm, which will use :

1. Population of solutions (rules)
2. Parent selection method (Roulette wheel)
3. Multi-point Crossover operator (Multi-point Crossover)
4. Mutation operator (Mutation)
5. Elitism
6. Variable termination criteria
7. User-defined parameters In this work, the effectiveness of CS in application problems will be studied for various values of the parameters.

2. Περιγραφή του θέματος και των απαιτήσεών του.

2.1.1 Θέμα της Εργασίας.

Η παρούσα εργασία έχει ως στόχο την υλοποίηση ενός Classifier System (CS) με σταθερό πλήθος κανόνων, το οποίο θα διαθέτει δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (rule chaining) για εφαρμογή σε προβλήματα κατηγοριοποίησης (classification). Το σύστημα θα είναι σε θέση να εξελίσσει σετ από κανόνες τύπου “if-then” χρησιμοποιώντας γενετικούς αλγορίθμους (Genetic Algorithms – GA), με σκοπό την προσομοίωση επιθυμητής συμπεριφοράς εισόδου-εξόδου.

Η επιλογή ενός CS με σταθερό πλήθος κανόνων επιτρέπει τον έλεγχο και την αξιολόγηση της απόδοσης του συστήματος σε διαφορετικά προβλήματα, ενώ η δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων προσφέρει τη δυνατότητα συμπερασμού μέσω πολλαπλών κανόνων, ενισχύοντας τη γενικότητα και την προσαρμοστικότητα του συστήματος.

2.1.2 Απαιτήσεις Υλοποίησης.

Η ανάπτυξη του CS περιλαμβάνει τα παρακάτω κύρια χαρακτηριστικά και λειτουργίες:

1. Σταθερό Πλήθος Κανόνων:

- Το CS θα διατηρεί έναν προκαθορισμένο αριθμό κανόνων κατά την εκτέλεση του.
- Κάθε κανόνας θα περιλαμβάνει **συνθήκη (condition)** και **ενέργεια (action)**, μαζί με τη δύναμή του (rule strength).

2. Αλγόριθμος Απονομής Βαθμών – Bucket Brigade:

- Οι κανόνες θα αξιολογούνται μέσω του **Bucket Brigade Algorithm**, ο οποίος κατανέμει «δυνάμεις» από κανόνες που

εκτελούνται σε άλλους κανόνες που συμμετέχουν στον συμπερασμό, προσομοιώνοντας την ανταμοιβή της επιτυχούς δράσης.

3. Γενετική Εξέλιξη Κανόνων:

- Το σύστημα θα χρησιμοποιεί **Γενετικό Αλγόριθμο (GA)** για την εξελικτική βελτίωση των κανόνων, με τα εξής χαρακτηριστικά:

1. **Πληθυσμός Λύσεων:** Αντιπροσωπεύει το σετ κανόνων που εξελίσσεται.
2. **Μέθοδος Επιλογής Γονέων:** Τροχός της Ρουλέτας (Roulette Wheel Selection) βάσει της δύναμης των κανόνων.
3. **Τελεστής Ανασυνδυασμού (Crossover):** Multi-point crossover για παραγωγή απογόνων.
4. **Τελεστής Μετάλλαξης (Mutation):** Τυχαία τροποποίηση κανόνων για εισαγωγή ποικιλίας.
5. **Ελιτισμός (Elitism):** Διατήρηση των καλύτερων κανόνων σε κάθε γενιά.
6. **Μεταβλητά Κριτήρια Τερματισμού:** Ορισμός από τον χρήστη για ολοκλήρωση της εξέλιξης (π.χ., μέγιστος αριθμός γενεών, επίτευξη στόχου ακρίβειας).
7. **Ρυθμιζόμενες Παράμετροι:** Παράμετροι του GA (πληθυσμός, πιθανότητα crossover, πιθανότητα mutation) καθοριζόμενες από τον χρήστη.

4. Δυνατότητα Αλυσιδωτής Ενεργοποίησης Κανόνων:

- Το CS θα μπορεί να ενεργοποιεί διαδοχικά κανόνες, όπου η έξοδος ενός κανόνα μπορεί να επηρεάσει την ενεργοποίηση άλλων κανόνων, επιτρέποντας **σύνθετο συμπερασμό**.

5. Εφαρμογή σε Προβλήματα Κατηγοριοποίησης:

- Το σύστημα θα αξιολογηθεί σε απλό πρόβλημα κατηγοριοποίησης, με στόχο την **ακρίβεια πρόβλεψης κατηγοριών** βάσει εισόδων.
- Η αποτελεσματικότητα του CS θα μελετηθεί για διάφορες τιμές παραμέτρων, παρέχοντας αναλυτικά αποτελέσματα και συμπεράσματα.

3. Σύντομη εισαγωγή στους Εξελικτικούς Αλγόριθμους, τους Γενετικούς Αλγόριθμους και τα Classifier Systems - αναφορά στη μηχανική μάθηση.

Οι εξελικτικοί αλγόριθμοι (EA), οι γενετικοί αλγόριθμοι (ΓΑ) και τα συστήματα ταξινόμησης (Classifier Systems) είναι τομείς της υπολογιστικής νοημοσύνης, εμπνευσμένοι από τη βιολογική εξέλιξη. Αποτελούν ισχυρά εργαλεία στη μηχανική μάθηση, ιδιαίτερα για προβλήματα βελτιστοποίησης και εξεύρεσης λύσεων σε πολύπλοκα περιβάλλοντα.

3.1 Εξελικτικοί Αλγόριθμοι (EA)

Οι Εξελικτικοί Αλγόριθμοι είναι μια ευρύτερη κατηγορία αλγορίθμων που μιμούνται τις διαδικασίες της φυσικής επιλογής. Ξεκινώντας με έναν τυχαίο αρχικό πληθυσμό υποψήφιων λύσεων, οι EA βελτιώνουν

σταδιακά τις λύσεις μέσω επαναληπτικών διαδικασιών, οι οποίες περιλαμβάνουν:

- **Επιλογή:** Οι καλύτερες λύσεις (με βάση μια συνάρτηση καταλληλότητας) επιλέγονται για να επιβιώσουν και να αναπαραχθούν.
- **Αναπαραγωγή:** Οι επιλεγμένες λύσεις συνδυάζονται (διασταύρωση) και μεταλλάσσονται για να δημιουργήσουν μια νέα γενιά λύσεων.
- **Αντικατάσταση:** Η νέα γενιά αντικαθιστά την προηγούμενη, και η διαδικασία επαναλαμβάνεται μέχρι να επιτευχθεί μια ικανοποιητική λύση.

3.2 Γενετικοί Αλγόριθμοι (ΓΑ)

Οι Γενετικοί Αλγόριθμοι είναι ο πιο γνωστός τύπος Εξελικτικών Αλγορίθμων. Χρησιμοποιούν μια αναπαράσταση των λύσεων ως "χρωμοσώματα" και εφαρμόζουν "γενετικούς" τελεστές (διασταύρωση και μετάλλαξη) για την παραγωγή νέων, βελτιωμένων λύσεων. Οι ΓΑ είναι ιδιαίτερα αποτελεσματικοί σε προβλήματα βελτιστοποίησης και αναζήτησης σε χώρους που είναι πολύ μεγάλοι για να εξερευνηθούν εξαντλητικά.

3.3 Συστήματα Ταξινόμησης (Classifier Systems)

Τα Συστήματα Ταξινόμησης, ή LCS (Learning Classifier Systems), αποτελούν ένα παράδειγμα μηχανικής μάθησης που συνδυάζει εξελικτικούς αλγορίθμους με μάθηση ενίσχυσης (reinforcement

learning). Ένα LCS λειτουργεί με βάση ένα σύνολο κανόνων (ταξινομητών), συνήθως με τη μορφή "αν-τότε" (if-then), οι οποίοι προσαρμόζονται και εξελίσσονται.

- **Λειτουργία:** Το σύστημα λαμβάνει εισόδους από το περιβάλλον και ενεργοποιεί τους αντίστοιχους κανόνες.
- **Ενίσχυση:** Η επιτυχία ή αποτυχία κάθε κανόνα ενισχύεται ή αποδυναμώνεται με βάση την ανάδραση από το περιβάλλον.
- **Εξέλιξη:** Ένας γενετικός αλγόριθμος χρησιμοποιείται για την ανακάλυψη νέων κανόνων και τη βελτίωση των υπαρχόντων.

3.4 Σχέση με τη Μηχανική Μάθηση

Οι εξελικτικοί αλγόριθμοι και, κατ' επέκταση, οι γενετικοί αλγόριθμοι και τα συστήματα ταξινόμησης, αποτελούν σημαντικό κομμάτι της μηχανικής μάθησης, ειδικά στην εξελικτική μάθηση. Χρησιμοποιούνται για:

- Βελτιστοποίηση παραμέτρων: Εύρεση των βέλτιστων παραμέτρων σε μοντέλα μηχανικής μάθησης.
- Εύρεση κανόνων: Ανακάλυψη κανόνων για ταξινόμηση ή πρόβλεψη, όπως συμβαίνει στα LCS.
- Αρχιτεκτονική νευρωνικών δικτύων: Βελτιστοποίηση της δομής των τεχνητών νευρωνικών δικτύων.
- Επίλυση πολύπλοκων προβλημάτων: Αντιμετώπιση προβλημάτων όπου ο χώρος των λύσεων είναι μεγάλος ή δύσκολος να εξερευνηθεί με παραδοσιακές μεθόδους.

4. Τι είναι και πώς λειτουργεί ένα Classifier System (CS)

4.1 Εισαγωγή

Τα Classifier Systems (CS) αποτελούν μια σημαντική κατηγορία συστημάτων μάθησης βασισμένων σε κανόνες που ενσωματώνουν τεχνικές εξελικτικών αλγορίθμων και ενισχυτικής μάθησης (**Reinforcement Learning**). Το πρώτο πλήρες CS προτάθηκε από τον John Holland τη δεκαετία του 1970, και έκτοτε έχουν εξελιχθεί σε εργαλεία για την επίλυση προβλημάτων προσαρμοστικής μάθησης και λήψης αποφάσεων σε δυναμικά περιβάλλοντα (Holland, 1975; Holland & Reitman, 1978).

Η βασική ιδέα των Classifier Systems είναι ότι το σύστημα "μαθαίνει" ποιες ενέργειες πρέπει να εκτελεί σε διαφορετικές καταστάσεις του περιβάλλοντος, χρησιμοποιώντας ένα σύνολο κανόνων της μορφής:



Οι κανόνες αυτοί εξελίσσονται και προσαρμόζονται ώστε να μεγιστοποιούν τη συνολική ανταμοιβή που μπορεί να λάβει το σύστημα.

4.2 Αρχιτεκτονική ενός Classifier System

Ένα τυπικό CS αποτελείται από τα εξής βασικά συστατικά (Holland, 1992; Wilson, 1995):

4.2.1 Μηχανισμός Αναγνώρισης (Matching)

Ο μηχανισμός αυτός ελέγχει ποιοι κανόνες είναι **σχετικοί** με την τρέχουσα κατάσταση του περιβάλλοντος. Το σύνολο των κανόνων που ταιριάζουν ονομάζεται **match set**. Μόνο οι κανόνες αυτού του συνόλου συμμετέχουν στην επιλογή ενέργειας.

4.2.2 Επιλογή Ενέργειας (Action Selection)

Από το match set επιλέγεται η ενέργεια που θα εκτελεστεί. Η επιλογή γίνεται συνήθως με βάση την **εκτίμηση της αποτελεσματικότητας κάθε κανόνα**, η οποία ενημερώνεται συνεχώς ανάλογα με την απόδοσή του.

4.2.3 Μηχανισμός Ανταμοιβής (Reinforcement Learning)

Οι κανόνες ενημερώνουν τις εκτιμήσεις τους βάσει της **ανταμοιβής που λαμβάνει το σύστημα** από το περιβάλλον. Η διαδικασία αυτή μπορεί να βασιστεί σε αλγορίθμους ενισχυτικής μάθησης, όπως το **Q-learning** ή **temporal difference learning**.

4.2.4 Γενετικός Αλγόριθμος (Genetic Algorithm)

Ένας **γενετικός αλγόριθμος (GA)** εφαρμόζεται περιοδικά στους κανόνες για να δημιουργεί νέους ή να βελτιώνει υπάρχοντες κανόνες. Η διαδικασία περιλαμβάνει:

- **Επιλογή:** Επιλογή των καλύτερων κανόνων με βάση την αξιολόγηση (**fitness**).
- **Διασταύρωση (crossover):** Συνδυασμός κανόνων για παραγωγή απογόνων.
- **Μετάλλαξη (mutation):** Τυχαίες τροποποιήσεις για εισαγωγή ποικιλίας.

Ο συνδυασμός **μηχανισμών μάθησης** και **γενετικού αλγορίθμου** κάνει το **CS ιδιαίτερα προσαρμοστικό** σε δυναμικά και περίπλοκα περιβάλλοντα.

4.3 Λειτουργία ενός CS – Βήμα προς Βήμα

Η λειτουργία ενός CS μπορεί να περιγραφεί συνοπτικά ως εξής (Holland & Reitman, 1978; Wilson, 1995):

1. Το σύστημα παρατηρεί την τρέχουσα κατάσταση του περιβάλλοντος.
2. Εντοπίζει όλους τους κανόνες που ταιριάζουν με την κατάσταση (match set).

3. Επιλέγει την ενέργεια με βάση την εκτίμηση της απόδοσης των ταιριαστών κανόνων.
4. Εκτελεί την επιλεγμένη ενέργεια.
5. Λαμβάνει ανταμοιβή (reward) από το περιβάλλον.
6. Ενημερώνει τις εκτιμήσεις των κανόνων που συμμετείχαν στην απόφαση.
7. Περιοδικά, ο γενετικός αλγόριθμος ανανεώνει τους κανόνες για βελτιστοποίηση της απόδοσης.

Ο συνδυασμός αυτών των μηχανισμών καθιστά τα CS συστήματα **αυτοπροσαρμοζόμενα**, ικανά να μαθαίνουν και να εξελίσσονται με το χρόνο.

4.4 Είδη Classifier Systems

Τα CS διακρίνονται σε διάφορους τύπους, ανάλογα με τον τρόπο μάθησης και εκπροσώπησης των κανόνων (Wilson, 1995):

1. Michigan-Style CS: Κάθε κανόνας αντιπροσωπεύει μια πιθανή λύση και εξελίσσεται ανεξάρτητα.
2. Pittsburgh-Style CS: Το σύνολο των κανόνων αποτελεί τη λύση, και εξελίσσεται ολόκληρο το σύνολο.
3. XCS (eXtended Classifier System): Μια βελτιωμένη έκδοση που εστιάζει σε ακριβείς προβλέψεις και σταθερή μάθηση.

Η πιο δημοφιλής σύγχρονη μορφή είναι το XCS, το οποίο χρησιμοποιεί fitness based on prediction accuracy αντί για επιβράβευση συνολικής απόδοσης, οδηγώντας σε πιο σταθερά και ακριβή συστήματα (Wilson, 1995).

4.5 Εφαρμογές

Τα CS έχουν εφαρμογές σε πολλούς τομείς της τεχνητής νοημοσύνης και της επιστήμης υπολογιστών, όπως:

- Προσαρμοστικά συστήματα ελέγχου (adaptive control systems).
- Ρομποτική και αυτόνομη πλοήγηση.
- Ανάλυση δεδομένων και εξόρυξη γνώσης (data mining).
- Προβλέψεις και συστήματα σύστασης (predictive modeling, recommendation systems).

Η δυνατότητα των CS να προσαρμόζονται και να εξελίσσονται τους καθιστά χρήσιμους σε περιβάλλοντα όπου οι συνθήκες αλλάζουν δυναμικά.

4.6 Πλεονεκτήματα και Μειονεκτήματα

Πλεονεκτήματα:

- Προσαρμοστικότητα σε δυναμικά περιβάλλοντα.
- Ικανότητα εκμάθησης πολύπλοκων σχέσεων κατάστασης-ενέργειας.
- Συνδυασμός ενισχυτικής μάθησης και εξελικτικής βελτιστοποίησης.

Μειονεκτήματα:

- Απαιτεί αρκετό χρόνο για εκπαίδευση.
- Πολύπλοκος χειρισμός σε μεγάλα σύνολα κανόνων.
- Η απόδοση εξαρτάται από τις παραμέτρους του γενετικού αλγορίθμου και της μάθησης.

4.7 Συμπεράσματα

Τα **Classifier Systems** αποτελούν ένα ισχυρό εργαλείο για την ανάπτυξη προσαρμοστικών και εξελικτικών συστημάτων μάθησης. Η συνδυαστική χρήση κανόνων, ενισχυτικής μάθησης και γενετικών αλγορίθμων τα καθιστά κατάλληλα για ένα μεγάλο εύρος εφαρμογών σε προβλήματα όπου η περιβαλλοντική δυναμική και η αβεβαιότητα είναι κρίσιμες.

Η σύγχρονη εξέλιξη των **CS**, όπως το **XCS**, δείχνει ότι αυτά τα συστήματα μπορούν να γίνουν ακόμα πιο ακριβή, αποδοτικά και σταθερά, καθιστώντας τα σημαντικό εργαλείο στη μελέτη της τεχνητής νοημοσύνης και της εξελικτικής υπολογιστικής.

5. Περιγραφή του συστήματος ταξινόμησης (CS) και του προβλήματος που θα αντιμετωπίσει.

Το έργο αυτό επικεντρώνεται στο σχεδιασμό, την υλοποίηση και την αξιολόγηση ενός **συστήματος ταξινόμησης (CS)** που είναι ικανό να εξελίσσεται και να συλλογίζεται μέσω της αλληλεπίδρασης ενός συνόλου κανόνων.

Το σύστημα έχει σχεδιαστεί για να συνδυάζει την **ενισχυτική μάθηση** και τους **γενετικούς αλγόριθμους (GA)** σε ένα ενοποιημένο υπολογιστικό πλαίσιο για την προσομοίωση προσαρμοστικής, ευφυούς συμπεριφοράς.

Ο κύριος στόχος είναι η κατασκευή ενός συστήματος ταξινόμησης με σταθερό αριθμό κανόνων που μπορούν συλλογικά να μάθουν να εκτελούν εργασίες ταξινόμησης συγκεκριμένα, την εργασία προσομοίωσης της συμπεριφοράς ενός **πολυπλέκτη 4 προς 1**.

Το **σύστημα ταξινόμησης** που αναπτύχθηκε σε αυτό το έργο λειτουργεί ως μοντέλο προσαρμοστικής μάθησης και συλλογιστικής, όπου ένας πληθυσμός κανόνων ανταγωνίζεται, συνεργάζεται και εξελίσσεται με βάση τη συμβολή του στην επίλυση ενός δεδομένου προβλήματος. Κάθε

κανόνας στον πληθυσμό αντιπροσωπεύεται από ένα ζεύγος συνθήκης-δράσης.

Το μέρος της συνθήκης ορίζει το μοτίβο εισόδου που μπορεί να αναγνωρίσει ο κανόνας, ενώ η δράση καθορίζει την έξοδο ή την απόκριση που παράγει ο κανόνας. Κάθε κανόνας συνδέεται με έναν αριθμητικό παράμετρο που ονομάζεται ισχύς, ο οποίος αντιπροσωπεύει τη σχετική χρησιμότητα ή «πιστότητα» του κανόνα στο σύστημα.

Με την πάροδο του χρόνου, αυτές οι δυνάμεις ενημερώνονται μέσω μιας διαδικασίας γνωστής ως αλγόριθμος **Bucket Brigade**, και οι ίδιοι οι κανόνες εξελίσσονται μέσω ενός **γενετικού αλγορίθμου** για τη βελτίωση της συνολικής απόδοσης του συστήματος.

5.1. Η αρχιτεκτονική του συστήματος ταξινόμησης (ClassifierSystem).

Το **σύστημα ταξινόμησης** που αναπτύσσεται αποτελείται από διάφορα βασικά στοιχεία που συνεργάζονται για να επιτρέψουν την προσαρμοστική μάθηση:

5.1.1 Πληθυσμός κανόνων (Population of Rules).

Το σύστημα διατηρεί έναν σταθερό πληθυσμό κανόνων, καθένας από τους οποίους ορίζεται από μια δυαδική συνθήκη και μια σχετική ενέργεια. Οι **συνθήκες** μπορεί να περιέχουν σύμβολα «δεν έχει σημασία» (#) για να επιτρέπουν τη γενίκευση σε διαφορετικά μοτίβα εισόδου. Για παράδειγμα, μια συνθήκη κανόνα όπως **10#0#1** αντιπροσωπεύει ένα μοτίβο που μπορεί να ταιριάζει με πολλαπλές εισόδους που ταιριάζουν στην ίδια δομή. Ο πληθυσμός κανόνων αντιπροσωπεύει τη συλλογική γνώση του συστήματος και κάθε κανόνας ανταγωνίζεται για να επηρεάσει τις αποφάσεις του συστήματος.

5.1.2 Αντιστοίχιση και ενεργοποίηση κανόνων (Rule Matching and Activation).

Όταν το **σύστημα** λαμβάνει μια είσοδο, αναζητά όλους τους κανόνες των οποίων οι συνθήκες ταιριάζουν με αυτήν την είσοδο. Η αντιστοίχιση είναι ευέλικτη το σύμβολο «δεν έχει σημασία» επιτρέπει στους κανόνες

να ταιριάζουν με μια σειρά εισόδων. Μεταξύ **όλων των κανόνων** που ταιριάζουν, το σύστημα επιλέγει αυτόν ή αυτούς με τη μεγαλύτερη ισχύ για να καθορίσει την ενέργεια που θα εκτελεστεί. Αυτή η διαδικασία λήψης αποφάσεων μιμείται τη συλλογιστική ή την «ενεργοποίηση αλυσίδας κανόνων», όπου πολλοί κανόνες μπορούν να επηρεάσουν την ενεργοποίηση ο ένας του άλλου με την πάροδο του χρόνου.

5.1.3 Ενίσχυση μέσω του αλγορίθμου Bucket

Brigade(Reinforcement via the Bucket Brigade Algorithm).

Μόλις το σύστημα παράγει ένα αποτέλεσμα, το συγκρίνει με το επιθυμητό αποτέλεσμα. Οι σωστοί κανόνες ανταμείβονται με την αύξηση της ισχύος τους, ενώ οι λανθασμένοι τιμωρούνται ή χάνουν μέρος της ισχύος τους. Ο μηχανισμός ανταμοιβής εφαρμόζεται χρησιμοποιώντας τον αλγόριθμο **Bucket Brigade**, όπου οι κανόνες διεκδικούν αποτελεσματικά το δικαίωμα να ενεργήσουν, και οι ανταμοιβές ή οι ποινές τους εξαρτώνται από την απόδοσή τους και τη συμβολή τους σε επιτυχημένες αποφάσεις. Αυτή η δυναμική προσαρμογή των δυνάμεων επιτρέπει στο σύστημα να ενισχύει χρήσιμες συμπεριφορές και να εξαλείφει σταδιακά τις αναποτελεσματικές.

5.1.4 Γενετικός αλγόριθμος για την εξέλιξη (Genetic Algorithm for Evolution).

Για να αποφευχθεί η στασιμότητα και να ενθαρρυνθεί η εξερεύνηση νέων συνδυασμών κανόνων, το σύστημα χρησιμοποιεί έναν γενετικό αλγόριθμο. Ο **γενετικός αλγόριθμος** εξελίσσει περιοδικά τον πληθυσμό των κανόνων με:

- **Επιλογή (Selection)** : Επιλογή γονικών κανόνων με βάση την πιθανότητα ανάλογα με τη δύναμή τους (**επιλογή ρουλέτας / Roulette Wheel Selection**).
- **Διασταύρωση (Crossover)** : Συνδυασμός τμημάτων δύο γονικών κανόνων μέσω μιας διασταύρωσης πολλαπλών σημείων (**multi-point crossover**) για τη δημιουργία κανόνων απογόνων που κληρονομούν ιδιότητες και από τους δύο γονείς.
- **Μετάλλαξη (Mutation)** : Τυχαία αναστροφή bit ή αλλαγή συμβόλων εντός των συνθηκών και των ενεργειών για την εισαγωγή ποικιλομορφίας και την ανακάλυψη νέων προτύπων.
- **Ελιτισμός (Elitism)** : Διατήρηση ενός υποσυνόλου των κανόνων με την καλύτερη απόδοση για να διασφαλιστεί ότι η πολύτιμη γνώση δεν θα χαθεί στις επόμενες γενιές (**generations**).

Αυτή η εξελικτική διαδικασία επιτρέπει στο σύστημα ταξινόμησης να προσαρμόζει τον εσωτερικό πληθυσμό κανόνων του για να προσεγγίζει καλύτερα τη σωστή αντιστοίχιση εισόδου-εξόδου με την πάροδο του χρόνου.

5.2 Το πρόβλημα: Εκμάθηση του πολυπλέκτη (Multiplexer) 4 προς 1.

Το πρόβλημα ταξινόμησης που εξετάζεται σε αυτό το έργο βασίζεται στη συμπεριφορά ενός ψηφιακού πολυπλέκτη 4 προς 1 (MUX). Ένας

πολυπλέκτης 4 προς 1 είναι ένα συνδυαστικό λογικό κύκλωμα με έξι bit εισόδου και ένα bit εξόδου.

Τα **έξι bit εισόδου** χωρίζονται σε **δύο bit επιλογής (S1 και S0)** και **τέσσερα bit δεδομένων (D0, D1, D2, D3)**. Τα bit επιλογής καθορίζουν ποια από τις **τέσσερις εισόδους δεδομένων** θα μεταβιβαστούν στην έξοδο.

Η λειτουργία του πολυπλέκτη μπορεί να εκφραστεί ως εξής:

$$Y=D_i \text{ όπου } i=(S_1S_0)$$

Αυτό σημαίνει ότι τα δύο bit επιλογής καθορίζουν ποιο από τα τέσσερα bit δεδομένων θα εμφανιστεί στην έξοδο.

Για παράδειγμα:

- Όταν **S1S0 = 00**, η έξοδος ισούται με **D0**.
- Όταν **S1S0 = 01**, η έξοδος ισούται με **D1**.
- Όταν **S1S0 = 10**, η έξοδος ισούται με **D2**.
- Όταν **S1S0 = 11**, η έξοδος ισούται με **D3**.

Σε μορφή δυαδικής συμβολοσειράς, η είσοδος **S1S0D0D1D2D3**

αντιπροσωπεύει ένα **μοτίβο έξι bit** και το **σύστημα ταξινόμησης** πρέπει να μάθει να παράγει τη **σωστή έξοδο ενός bit Y**.

Για παράδειγμα:

- Είσοδος: **000000** → Έξοδος: **0** (Επιλογή **D0 = 0**).

- Είσοδος: 001000 → Έξοδος: 1 (Επιλογή **D0** = 1).
- Είσοδος: 010000 → Έξοδος: 0 (Επιλογή **D1** = 0).
- Είσοδος: 010100 → Έξοδος: 1 (Επιλογή **D1** = 1).
- Είσοδος: 100000 → Έξοδος: 0 (Επιλογή **D2** = 0).
- Είσοδος: 100010 → Έξοδος: 1 (Επιλογή **D2** = 1).

- Είσοδος: 110000 → Έξοδος: 0 (Επιλογή **D3** = 0).
- Είσοδος: 110001 → Έξοδος: 1 (Επιλογή **D3** = 1).

Αυτή η αντιστοίχιση αποτελεί το σύνολο εκπαίδευσης για το σύστημα ταξινόμησης. Το σύστημα πρέπει να μάθει την υποκείμενη λογική σχέση ότι τα δύο πρώτα bit επιλέγουν ποιο επόμενο bit καθορίζει την έξοδο αντί να απομνημονεύει κάθε **πιθανό ζεύγος εισόδου-εξόδου** ξεχωριστά.

5.3 Σκοπός και μαθησιακός στόχος

Ο πρωταρχικός σκοπός αυτής της εργασίας είναι να διερευνήσει πώς ένα **σύστημα ταξινόμησης** μπορεί να μάθει αυτόνομα τη λειτουργική συμπεριφορά ενός **πολυπλέκτη 4 προς 1**. Αυτό αποτελεί μια σημαντική πρόκληση, επειδή ο πολυπλέκτης ενσωματώνει μια μη γραμμική, υπό όρους σχέση μεταξύ των εισόδων και της εξόδου του. Σε αντίθεση με έναν απλό πίνακα αναζήτησης, το σύστημα ταξινόμησης πρέπει να αναπτύξει εσωτερικούς κανόνες που γενικεύουν τη λογική κατανοώντας ότι τα δύο πρώτα bit ελέγχουν ποιο από τα άλλα τέσσερα bit καθορίζει το αποτέλεσμα.

Συνεπώς, το έργο αυτό διερευνά:

- Πόσο αποτελεσματικά μπορεί το CS να αναπτύξει σύνολα κανόνων που αντιπροσωπεύουν σωστά τις υπό όρους, πολυμεταβλητές λογικές σχέσεις.
- Πώς διάφορες παράμετροι (όπως ο ρυθμός μετάλλαξης, οι τιμές ανταμοιβής, ο ρυθμός ελιτισμού και το μέγεθος του πληθυσμού) επηρεάζουν τη σύγκλιση, τη σταθερότητα και την ακρίβεια του συστήματος.
- Εάν ο εξελιγμένος πληθυσμός κανόνων μπορεί να γενικευτεί πέρα από τα συγκεκριμένα παραδείγματα στα δεδομένα εκπαίδευσης, σχηματίζοντας μια ερμηνεύσιμη βάση γνώσεων.
- Με τη μελέτη αυτής της **μαθησιακής διαδικασίας**, το έργο στοχεύει να αποδείξει την προσαρμοστική και αυτοοργανωτική φύση των συστημάτων ταξινόμησης και την ικανότητά τους να προσεγγίζουν τη συμβολική συλλογιστική.

5.4 Λειτουργία του αναπτυγμένου CS.

Κατά τη διάρκεια της εκπαίδευσης, το σύστημα επεξεργάζεται επαναληπτικά τα ζεύγη **εισόδου-εξόδου** από το σύνολο δεδομένων του πολυπλέκτη.

Για κάθε **μοτίβο εισόδου (input pattern)**:

- Το **σύστημα** προσδιορίζει όλους τους κανόνες των οποίων οι συνθήκες ταιριάζουν με την είσοδο.
- Ο **καταλληλότερος κανόνας (ή σύνολο κανόνων / or set of rules)** ενεργοποιείται και η δράση του καθορίζει την προβλεπόμενη έξοδο.

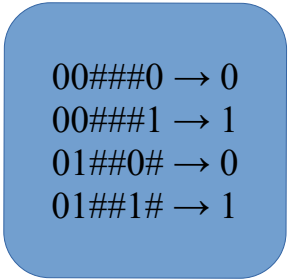
Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

- Το **σύστημα** συγκρίνει την προβλεπόμενη έξοδο με την πραγματική έξοδο.
- Η **ισχύς του κανόνα (rule's strength)** ενημερώνεται αυξάνεται αν είναι σωστή, μειώνεται αν όχι σύμφωνα με το σύστημα ανταμοιβών και ποινών που ορίζεται στον μηχανισμό **Bucket Brigade**.

Περιοδικά, ο γενετικός αλγόριθμος εξελίσσει τον πληθυσμό, βελτιώνοντας τη συνολική βάση γνώσεων.

Αυτή η διαδικασία επαναλαμβάνεται σε πολλές γενιές έως ότου ο πληθυσμός του ταξινομητή σταθεροποιηθεί και παράγει σταθερά υψηλή ακρίβεια ταξινόμησης.

Μέσω της εξέλιξης, το **CS** μαθαίνει συμπαγείς και γενικευμένους κανόνες όπως:



```
00###0 → 0
00###1 → 1
01##0# → 0
01##1# → 1
```

και ούτω καθεξής δείχνοντας ότι το σύστημα έχει μάθει το μοτίβο υπό όρους επιλογής του **πολυπλέκτη**.

5.5 Σημασία της έρευνας.

Η σημασία της ανάπτυξης αυτού του συστήματος ταξινόμησης έγκειται στην ικανότητά του να απεικονίζει τον τρόπο με τον οποίο η μάθηση βάσει κανόνων και οι εξελικτικοί μηχανισμοί μπορούν να μιμηθούν τη γνωστική συλλογιστική.

Αν και ο **πολυπλέκτης 4 προς 1** φαίνεται απλός, αποτελεί ένα εξαιρετικό σημείο αναφοράς για την αξιολόγηση των προσαρμοστικών συστημάτων μάθησης. Το πρόβλημα συνδυάζει την υπό όρους λογική, την αναγνώριση προτύπων και τη γενίκευση όλα θεμελιώδη στοιχεία της έξυπνης λήψης αποφάσεων.

Αυτή η εφαρμογή καταδεικνύει επίσης πώς τα υβριδικά μοντέλα μάθησης που συνδυάζουν την ενισχυτική μάθηση (απόδοση πιστώσεων) και την εξελικτική αναζήτηση (**GA**) μπορούν να συγκλίνουν προς βέλτιστες στρατηγικές συμπεριφοράς. Το πλαίσιο σταθερών κανόνων εξασφαλίζει υπολογιστικό έλεγχο και συνέπεια, επιτρέποντας παράλληλα στο σύστημα να εξελίσσεται εντός καθορισμένων δομικών ορίων.

Επιπλέον, το έργο θα επιτρέψει τον πειραματισμό με διάφορες παραμέτρους που ορίζονται από τον χρήστη:

- **Μέγεθος πληθυσμού (Population size)**
- **Ρυθμός μετάλλαξης (Mutation rate)**
- **Δύναμη ανταμοιβής και τιμωρίας (Reward and penalty strengths)**
- **Αναλογία ελιτισμού (Elitism proportion)**

- **Φορολογικός συντελεστής (απόσβεση δύναμης) Taxation rate (strength decay)**
- **Κριτήρια τερματισμού (Termination criteria)**

Αυτές οι παράμετροι επηρεάζουν άμεσα την ταχύτητα σύγκλισης, την ποικιλομορφία των κανόνων και τη σταθερότητα της μάθησης, παρέχοντας πολύτιμες πληροφορίες για τον τρόπο με τον οποίο αλληλεπιδρούν οι εξελικτικές και οι ενισχυτικές διαδικασίες στα προσαρμοστικά συστήματα.

5.6 Αναμενόμενα αποτελέσματα.

Μέχρι το τέλος του έργου, το σύστημα ταξινόμησης αναμένεται να:

- Μάθει με επιτυχία την αντιστοίχιση του **πολυπλέκτη 4 προς 1 (4-to-1 multiplexer)** μέσω προσαρμοστικής εξέλιξης.
- Αναπτύξει ένα συμπαγές σύνολο ισχυρών, γενικευμένων κανόνων που αποτυπώνουν τις σωστές λογικές σχέσεις.
- Επιδείξει ανθεκτικότητα στη διακύμανση των παραμέτρων, δείχνοντας σύγκλιση προς υψηλή ακρίβεια σε πολλαπλές εκτελέσεις.
- Παράγει ερμηνεύσιμα σύνολα κανόνων που μπορούν να εξεταστούν για την κατανόηση της γνώσης που έχει αποκτήσει το

σύστημα.

- Χρησιμεύει ως θεμελιώδες μοντέλο για την επέκταση των **συστημάτων ταξινόμησης (CSs)** σε πιο σύνθετα προβλήματα ταξινόμησης και ομαδοποίησης.

Επιπλέον, αυτή η εφαρμογή θα παρέχει μια σαφή επίδειξη της ενεργοποίησης του κανόνα της αλυσίδας, όπου η έξοδος ενός κανόνα μπορεί να επηρεάσει έμμεσα την ενεργοποίηση ενός άλλου μιμούμενη την ιεραρχική διαδικασία συλλογιστικής που παρατηρείται στην ανθρώπινη γνωστική λειτουργία και στα συμβολικά συστήματα **τεχνητής νοημοσύνης (AI systems)**.

5.7 Ευρύτερο πλαίσιο και εφαρμογές.

Αν και το άμεσο πρόβλημα που επιλύεται είναι η ταξινόμηση του πολυπλέκτη, το πλαίσιο που αναπτύχθηκε εδώ είναι ευρέως εφαρμόσιμο σε άλλα προβλήματα αναγνώρισης προτύπων και ομαδοποίησης. Το πρότυπο του συστήματος ταξινόμησης δεν περιορίζεται στα ψηφιακά κυκλώματα, αλλά μπορεί να επεκταθεί σε οποιονδήποτε τομέα που απαιτεί:

- **Λήψη αποφάσεων υπό αβεβαιότητα,**
- **Ανακάλυψη προσαρμοστικών κανόνων,**
- **Εξέλιξη συμβολικής γνώσης,**

- ή ταξινόμηση σύνθετων, πολυδιάστατων δεδομένων.

Τέτοια συστήματα έχουν πιθανή χρήση σε αυτόνομους πράκτορες, εξόρυξη δεδομένων, ρομποτικό έλεγχο, διαγνωστικά συστήματα και έξυπνη υποστήριξη λήψης αποφάσεων, όπου είναι επιθυμητά ευέλικτα, ερμηνεύσιμα και εξελισσόμενα μοντέλα βασισμένα σε κανόνες.

5.8 Συμπέρασμα.

Συνοπτικά, το έργο θα αναπτύξει ένα πλήρες **σύστημα ταξινόμησης (CS)** με σταθερό αριθμό κανόνων, ενσωματώνοντας τόσο τον αλγόριθμο **Bucket Brigade** για ενισχυτική μάθηση όσο και τον **γενετικό αλγόριθμο (Genetic Algorithm)** για **εξελικτική βελτιστοποίηση (Reinforcement learning)**. Το **σύστημα** θα εκπαιδευτεί και θα δοκιμαστεί στο πρόβλημα του **πολυπλέκτη 4 προς 1**, αποδεικνύοντας την ικανότητά του να εξελίσσεται και να βελτιώνει έναν **πληθυσμό κανόνων** που εκτελούν συλλογικά ακριβή ταξινόμηση.

Μέσω **πειραματισμού, ανάλυσης και ρύθμισης παραμέτρων**, το έργο αυτό θα δείξει πώς τα συστήματα ταξινόμησης μπορούν να προσομοιώσουν τη **μάθηση**, την προσαρμογή και τη συλλογιστική — βασικές ιδιότητες των **ευφύων υπολογιστικών μοντέλων**. Με την εξειδίκευση στην εργασία του πολυπλέκτη, το αναπτυγμένο **CS** θα χρησιμεύσει τόσο ως απόδειξη της ιδέας όσο και ως βάση για την επέκταση των **προσαρμοστικών συστημάτων** που βασίζονται σε κανόνες σε πιο σύνθετους τομείς **ομαδοποίησης και ταξινόμησης**.

6 Τεχνική περιγραφή του κώδικα που αναπτύχθηκε.

Ο κώδικας που αναπτύχθηκε υλοποιεί ένα πλήρες **σύστημα ταξινόμησης (CS)** σχεδιασμένο για την επίλυση ενός προβλήματος ταξινόμησης **πολυπλέκτη 4 προς 1**, συνδυάζοντας την ενισχυτική μάθηση και τον εξελικτικό υπολογισμό. Είναι γραμμένο σε C++ και ακολουθεί μια αρθρωτή δομή, με σαφή διαχωρισμό των στοιχείων που χειρίζονται τη διαχείριση κανόνων, την αξιολόγηση καταλληλότητας, τις ενημερώσεις ενίσχυσης και τη γενετική εξέλιξη.

Το **σύστημα** έχει σχεδιαστεί με βάση την έννοια ενός σταθερού πληθυσμού κανόνων, όπου κάθε κανόνας αντιπροσωπεύει ένα πιθανό ζεύγος συνθηκών-ενεργειών. Το πρόγραμμα προσομοιώνει τη διαδικασία της προσαρμοστικής μάθησης μέσω της επαναλαμβανόμενης έκθεσης σε δεδομένα εκπαίδευσης, όπου οι κανόνες ανταγωνίζονται, συνεργάζονται και εξελίσσονται με βάση την προγνωστική τους απόδοση.

6.1 Επισκόπηση συστήματος.

Στον πυρήνα του, ο κώδικας μοντελοποιεί τρία κύρια λειτουργικά υποσυστήματα:

1. **Υποσύστημα διαχείρισης κανόνων** – υπεύθυνο για την αποθήκευση, την αρχικοποίηση, την αντιστοίχιση και την ενημέρωση των κανόνων.

2. Υποσύστημα ενισχυτικής μάθησης – εφαρμόζει τον **αλγόριθμο Bucket Brigade Algorithm (BBA)** για να επιβραβεύει ή να τιμωρεί τους κανόνες με βάση την απόδοση της ταξινόμησης.
3. Υποσύστημα **γενετικού αλγορίθμου** – διαχειρίζεται την εξέλιξη των κανόνων μέσω επιλογής, διασταύρωσης, μετάλλαξης και ελιτισμού, προκειμένου να διατηρηθεί η ποικιλομορφία και η προσαρμοστικότητα του πληθυσμού.

Κάθε ένα από αυτά τα υποσυστήματα λειτουργεί διαδραστικά εντός ενός κύριου βρόχου ελέγχου που επεξεργάζεται το σύνολο εκπαίδευσης, αξιολογεί τις προβλέψεις, ενημερώνει την ισχύ των κανόνων και εξελίσσει τον πληθυσμό των κανόνων σε πολλές γενιές.

6.2 Αναπαράσταση κανόνων.

Κάθε κανόνας στο σύστημα υλοποιείται ως δομημένο αντικείμενο που περιέχει τα ακόλουθα χαρακτηριστικά:

- **Συνθήκη (Condition)**: Μια δυαδική συμβολοσειρά (π.χ. «10#01#»), που αντιπροσωπεύει το μοτίβο των bit εισόδου που αναγνωρίζει ο κανόνας. Το σύμβολο # λειτουργεί ως τελεστής «δεν έχει σημασία», επιτρέποντας μερική γενίκευση σε όλες τις εισόδους.

- **Ενέργεια (Action)**: Ένα δυαδικό ψηφίο («0» ή «1») που υποδεικνύει την έξοδο που προβλέπει ο κανόνας όταν η συνθήκη του ταιριάζει με την είσοδο.
- **Ισχύς (Strength)**: Μια τιμή κινητής υποδιαστολής που αντιπροσωπεύει το συσσωρευμένο πιστωτικό ή επίπεδο απόδοσης του κανόνα. Η ισχύς ενημερώνεται συνεχώς μέσω ενίσχυσης.

Οι κανόνες αποθηκεύονται σε έναν διάνυσμα πληθυσμού σταθερού μεγέθους, ο οποίος αρχικοποιείται τυχαία στην αρχή της προσομοίωσης. Αυτή η δομή εξασφαλίζει τον υπολογιστικό έλεγχο, παρέχοντας παράλληλα επαρκή ποικιλομορφία για τις εξελικτικές διαδικασίες.

6.3 Δεδομένα εκπαίδευσης και ορισμός του προβλήματος.

Ο κώδικας χρησιμοποιεί ένα σύνολο δεδομένων εκπαίδευσης που κωδικοποιεί τη λειτουργία του **πολυπλέκτη 4 προς 1**. Κάθε είσοδος αποτελείται από έξι δυαδικά ψηφία που αντιπροσωπεύουν τα **S1, S0, D0, D1, D2, D3**, και κάθε έξοδος αντιστοιχεί στη γραμμή δεδομένων που επιλέγεται από τα bit ελέγχου (**S1S0**).

Παραδείγματα δειγμάτων εκπαίδευσης:

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```
{ "000000", "0" } → Select D0 = 0
{ "001000", "1" } → Select D0 = 1
{ "010000", "0" } → Select D1 = 0
{ "010100", "1" } → Select D1 = 1
{ "100000", "0" } → Select D2 = 0
{ "100010", "1" } → Select D2 = 1
{ "110000", "0" } → Select D3 = 0
{ "110001", "1" } → Select D3 = 1
```

Αυτά τα παραδείγματα χρησιμοποιούνται για την εκπαίδευση του συστήματος ταξινόμησης, επιτρέποντάς του να συμπεράνει την υποκείμενη λογική σχέση αντί να απομνημονεύει ρητά ζεύγη εισόδου-εξόδου.

6.4 Μηχανισμός αντιστοίχισης.

Όταν παρουσιάζεται ένα μοτίβο εισόδου, το σύστημα αναζητά όλους τους κανόνες των οποίων οι συνθήκες ταιριάζουν με την είσοδο.

Ένας κανόνας ταιριάζει με μια είσοδο εάν, για κάθε θέση bit:

- Το bit της συνθήκης ισούται με το bit της εισόδου, ή
- Το bit της συνθήκης είναι το σύμβολο «δεν έχει σημασία» #.

Όλοι οι κανόνες αντιστοίχισης συλλέγονται σε ένα σύνολο αντιστοίχισης. Εάν πολλοί κανόνες ταιριάζουν με την ίδια είσοδο, επιλέγεται αυτός με τη μεγαλύτερη ισχύ για να παράγει την έξοδο. Αυτό προσομοιώνει τον

ανταγωνισμό μεταξύ των κανόνων για τον έλεγχο της διαδικασίας λήψης αποφάσεων του συστήματος.

6.5 Αξιολόγηση και ενίσχυση της καταλληλότητας (αλγόριθμος Bucket Brigade).

Μόλις το σύστημα παράγει ένα προβλεπόμενο αποτέλεσμα, το συγκρίνει με το σωστό αποτέλεσμα από τα δεδομένα εκπαίδευσης.

Εάν το προβλεπόμενο αποτέλεσμα είναι σωστό, ο κανόνας που είναι υπεύθυνος λαμβάνει μια ανταμοιβή, αυξάνοντας τη δύναμή του.

Εάν η πρόβλεψη είναι λανθασμένη, η δύναμη του κανόνα τιμωρείται, μειώνοντας την επιρροή του σε μελλοντικές αποφάσεις.

Αυτή η διαδικασία ελέγχεται από παραμέτρους ενίσχυσης:

- **REWARD_VALUE** → μέγεθος της θετικής ενίσχυσης.
- **PENALTY_VALUE** → μέγεθος της τιμωρίας.
- **TAX_RATE** → συνεχής συντελεστής αποσβέσεως που εφαρμόζεται σε όλους τους κανόνες για την αποφυγή ανεξέλεγκτης συσσώρευσης ισχύος.

Αυτός ο μηχανισμός αποτελεί τον **αλγόριθμο Bucket Brigade (BBA)**, όπου η πίστωση κατανέμεται αναλογικά μεταξύ των ενεργών κανόνων, εξασφαλίζοντας ότι μόνο οι πιο συνεπείς και ακριβείς κανόνες διατηρούν υψηλή ισχύ με την πάροδο του χρόνου.

6.6 Εξελικτική μάθηση (γενετικός αλγόριθμος).

Για να βελτιώσει την προσαρμοστικότητα και να διατηρήσει την ποικιλομορφία, το σύστημα εφαρμόζει περιοδικά έναν **γενετικό αλγόριθμο (GA)** για να εξελίξει τον πληθυσμό των κανόνων.

Ο **GA** λειτουργεί με τα ακόλουθα βήματα:

1. Επιλογή (Selection):

Δύο γονικοί κανόνες επιλέγονται χρησιμοποιώντας την επιλογή **Roulette Wheel**, όπου η πιθανότητα επιλογής είναι ανάλογη με τη δύναμη κάθε κανόνα. Οι ισχυρότεροι κανόνες έχουν μεγαλύτερη πιθανότητα αναπαραγωγής.

2. Διασταύρωση (Crossover):

Οι επιλεγμένοι γονικοί κανόνες υποβάλλονται σε πολυσημειακή διασταύρωση, όπου τμήματα των συμβολοσειρών συνθηκών τους ανταλλάσσονται σε τυχαία σημεία διασταύρωσης για να δημιουργηθούν δύο απόγονοι.

Παράδειγμα:

```
Parent 1: 10#0#1
Parent 2: 11#1#0
Offspring 1: 10#1#0
Offspring 2: 11#0#1
```

3. Μετάλλαξη (Mutation):

Τα bits συνθηκών κάθε απογόνου υπόκεινται σε τυχαίες μεταλλάξεις με μικρή πιθανότητα (**MUTATION_RATE**).

Η μετάλλαξη εισάγει νέα σύμβολα (0, 1 ή #), αποτρέποντας την πρόωρη σύγκλιση.

4. Ελιτισμός (Elitism):

Ένα υποσύνολο των κανόνων με την καλύτερη απόδοση (με βάση τη δύναμη) διατηρείται χωρίς αλλαγές, ώστε να διασφαλιστεί ότι το σύστημα διατηρεί τις καλές λύσεις που έχει μάθει προηγουμένως.

5. Αντικατάσταση (Replacement):

Οι πιο αδύναμοι κανόνες στον πληθυσμό αντικαθίστανται από νεοδημιουργημένους απογόνους, διασφαλίζοντας ότι κάθε γενιά διατηρεί σταθερό μέγεθος πληθυσμού.

Αυτός ο εξελικτικός κύκλος επιτρέπει στο σύστημα ταξινόμησης να βελτιώνει προοδευτικά τη βάση κανόνων του, ανακαλύπτοντας πιο ακριβείς και γενικευμένες αντιστοιχίσεις συνθηκών-ενεργειών.

6.7 Ροή προσομοίωσης.

Η συνολική λειτουργία του συστήματος ακολουθεί τον ακόλουθο βρόχο:

1. Αρχικοποίηση (Initialization):

- ♦ Τυχαία δημιουργία του αρχικού πληθυσμού κανόνων.

- ♦ Αρχικοποίηση παραμέτρων όπως το μέγεθος του πληθυσμού, ο ρυθμός μετάλλαξης, οι τιμές ανταμοιβής/ποινής και το όριο γενιάς.

2. Κύκλος εκπαίδευσης (Training Cycle):

- ♦ Παρουσίαση κάθε εισόδου εκπαίδευσης στο σύστημα.
- ♦ Προσδιορισμός των κανόνων αντιστοίχισης και επιλογή του καλύτερου.
- ♦ Σύγκριση της προβλεπόμενης εξόδου με την αναμενόμενη έξοδο.
- ♦ Ενημερώστε τις δυνάμεις των κανόνων μέσω του αλγορίθμου Bucket Brigade.

3. Γενετική εξέλιξη:

- ♦ Μετά από κάθε κύκλο εκπαίδευσης (ή καθορισμένο αριθμό επαναλήψεων), ενεργοποιήστε τον γενετικό αλγόριθμο.
- ♦ Εκτελέστε επιλογή, διασταύρωση, μετάλλαξη και ελιτισμό.
- ♦ Αντικαταστήστε τους αδύναμους κανόνες με νέους απογόνους.

4. Αξιολόγηση και καταγραφή:

- ♦ Υπολογίστε την ακρίβεια του συστήματος (ποσοστό σωστά ταξινομημένων δειγμάτων).

5.

- ♦ Προσδιορίστε και αποθηκεύστε τον ελιτιστικό κανόνα (τον κανόνα με τη μεγαλύτερη ισχύ).
- ♦ Αποθηκεύστε στατιστικά στοιχεία προόδου (γενιά, ακρίβεια, ισχύς κανόνων) σε ένα αρχείο κειμένου με χρονική σήμανση.

6. Τερματισμός:

- ♦ Η διαδικασία επαναλαμβάνεται έως ότου πληρωθεί ένας καθορισμένος αριθμός γενεών ή κριτήρια σύγκλισης.
- ♦ Ο τελικός πληθυσμός κανόνων αντιπροσωπεύει το σύστημα ταξινόμησης που έχει μάθει και είναι ικανό να αναπαράγει τη λογική του πολυπλέκτη.

6.8 Έξοδος και αναφορά.

Στο τέλος της εκτέλεσης, το πρόγραμμα παράγει:

- Ένα λεπτομερές αρχείο αποτελεσμάτων (.txt) που περιέχει:
 - Παράμετροι συστήματος που χρησιμοποιήθηκαν (μέγεθος πληθυσμού, ρυθμός μετάλλαξης, ανταμοιβή/ποινή, ποσοστό ελιτισμού)
 - Τελικό σύνολο κανόνων με συνθήκη, δράση και ισχύ
 - Λεπτομέρειες κανόνων ελίτ
 - Στατιστικά στοιχεία ακρίβειας συστήματος ανά γενιά
- Μια σύνοψη κονσόλας που εμφανίζει την πρόοδο σύγκλισης και τα μέτρα απόδοσης.

Αυτή η έξοδος παρέχει τόσο ποσοτικές όσο και ποιοτικές πληροφορίες σχετικά με τον τρόπο εξέλιξης του συστήματος ταξινόμησης και τους κανόνες που κυριάρχησαν.

6.9 Παραμετροποίηση και έλεγχος χρήστη.

Ο κώδικας έχει σχεδιαστεί με πολλαπλές παραμέτρους που μπορούν να ρυθμιστούν από τον χρήστη και ορίζονται ως παγκόσμιες σταθερές ή διαμορφώσιμες εισόδους, επιτρέποντας τον πειραματισμό και την ανάλυση.

Οι βασικές παράμετροι περιλαμβάνουν:

Παράμετρος	Περιγραφή
POP_SIZE	Αριθμός κανόνων που διατηρούνται στο σύστημα.
MUTATION_RATE	Πιθανότητα μετάλλαξης bit στο GA.
CROSSOVER_POINTS	Αριθμός σημείων διασταύρωσης ανά αναπαραγωγή.
REWARD_VALUE	Μέγεθος ανταμοιβής για σωστές προβλέψεις.
PENALTY_VALUE	Μέγεθος ποινής για λανθασμένες προβλέψεις.
TAX_RATE	Συντελεστής εξασθένησης ισχύος που εφαρμόζεται σε όλους τους κανόνες.
ELITISM_RATE	Ποσοστό κανόνων που διατηρούνται σε κάθε γενιά NUM_GENERATIONS.
NUM_GENERATIONS	Συνολικοί εξελικτικοί κύκλοι πριν από τον τερματισμό.

Αυτή η δυνατότητα διαμόρφωσης επιτρέπει στον χρήστη να ρυθμίσει με ακρίβεια το σύστημα για διαφορετικές συμπεριφορές μάθησης, ανάλυση σταθερότητας και βελτιστοποίηση απόδοσης.

6.10 Τεχνικά χαρακτηριστικά.

- **Γλώσσα: C++** (υψηλή απόδοση και αντικειμενοστραφής δομή)
- **Δομές δεδομένων (Data Structures):** Διανύσματα για δυναμική διαχείριση πληθυσμού, συμβολοσειρές για χειρισμό δυαδικών συνθηκών
- **Αλγόριθμοι (Algorithms):**
 - ✦ Ενισχυτική μάθηση (Bucket Brigade)
 - ✦ Εξελικτική αναζήτηση (Γενετικός αλγόριθμος)
 - ✦ Επιλογή τροχού ρουλέτας
 - ✦ Πολυσημειακή διασταύρωση και μετάλλαξη
 - ✦ Αντικατάσταση κανόνων και ελιτισμός
- **Χειρισμός αρχείων (File Handling):**

Αυτόματη δημιουργία αρχείων καταγραφής με χρονική σήμανση για την αποθήκευση των αποτελεσμάτων των πειραμάτων.
- **Επεκτασιμότητα (Scalability):**

Η αρχιτεκτονική επιτρέπει την εύκολη επέκταση σε **μεγαλύτερους πολυπλέκτες** (π.χ. 8 προς 1, 16 προς 1) ή άλλες λογικές εργασίες ταξινόμησης.

7 Περιγραφή της ρύθμισης και των αποτελεσμάτων των εκτελεσθέντων κύκλων.

Προκειμένου να αξιολογηθεί η απόδοση του εφαρμοσμένου συστήματος ταξινόμησης (CS) στην εργασία του **πολυπλέκτη 4 προς 1**, πραγματοποιήθηκε μια εκτεταμένη σειρά πειραματικών κύκλων. Κάθε δοκιμή αντιστοιχεί σε μια πλήρη εκτέλεση του συστήματος, ξεκινώντας από έναν τυχαία αρχικοποιημένο πληθυσμό κανόνων και προχωρώντας σε έναν σταθερό αριθμό γενεών, κατά τη διάρκεια των οποίων ο μηχανισμός ενίσχυσης **Bucket Brigade** και ο **Γενετικός Αλγόριθμος (GA)** συνεργάζονται για την προσαρμογή του συνόλου των κανόνων.

Τα διαθέσιμα αρχεία αποτελεσμάτων (ένα ανά πείραμα) καταγράφουν τις ρυθμίσεις των παραμέτρων, τον τελικό πληθυσμό κανόνων, το υποσύνολο των κανόνων υψηλής ισχύος και την τελική ακρίβεια ταξινόμησης για κάθε δοκιμή. Συνολικά, πραγματοποιήθηκε μεγάλος αριθμός τέτοιων πειραμάτων (πάνω από 200 αρχεία αποτελεσμάτων), διερευνώντας τη συμπεριφορά του συστήματος υπό διαφορετικά μήκη εξέλιξης και, ενδεχομένως, διαφορετικές διαμορφώσεις παραμέτρων. Η παρακάτω περιγραφή συνθέτει τη ρύθμιση και τις παρατηρούμενες τάσεις σε όλες αυτές τις δοκιμές, χρησιμοποιώντας αντιπροσωπευτικά πειράματα με 2 000, 6 000 και 14 000 γενιές ως συγκεκριμένα παραδείγματα.

7.1 Πειραματική διάταξη.

7.1.1 Πρόβλημα και δεδομένα εκπαίδευσης.

Όλα τα πειράματα αντιμετωπίζουν το ίδιο πρόβλημα εποπτευόμενης μάθησης: την προσομοίωση ενός **πολυπλέκτη 4 προς 1** με έξι δυαδικές εισόδους και μία δυαδική έξοδο. Ο διάνυσμα εισόδου έχει τη μορφή

$$(S_1, S_0, D_0, D_1, D_2, D_3)$$

όπου (S_1, S_0) είναι bits επιλογής και (D_0, D_1, D_2, D_3) είναι εισόδους δεδομένων. Η έξοδος στόχος ορίζεται από

$$Y = D_i \text{ όπου } i = (S_1 S_0) \in \{00, 01, 10, 11\}$$

Το σύνολο εκπαίδευσης που χρησιμοποιείται από το CS (όπως είναι προγραμματισμένο στο πρόγραμμα) απαριθμεί και τις **64 πιθανούς συνδυασμούς** εισόδου μαζί με το σωστό bit εξόδου. Αυτό το σύνολο εκπαίδευσης είναι το ίδιο για κάθε εκτέλεση, εξασφαλίζοντας ότι οι διαφορές στην απόδοση προκύπτουν αποκλειστικά από τη στοχαστικότητα της διαδικασίας μάθησης και τις ρυθμίσεις των παραμέτρων, και όχι από διακυμάνσεις στα δεδομένα.

7.1.2 Βασική αλγοριθμική διαμόρφωση.

Η εφαρμογή διατηρεί έναν πληθυσμό κανόνων σταθερού μεγέθους και τους εξελίσσει μέσω επαναλαμβανόμενης εναλλαγής:

- **Ενίσχυση (Bucket Brigade)** : οι κανόνες υποβάλλουν προσφορές και λαμβάνουν ανταμοιβή/ποινή με βάση την ορθότητα της πρόβλεψης.
- **Γενετικός αλγόριθμος (Genetic Algorithm)** : οι κανόνες επιλέγονται, ανασυνδυάζονται και μεταλλάσσονται για να σχηματίσουν νέους υποψήφιους κανόνες, με τον ελιτισμό να διατηρεί ένα μέρος των καλύτερων.

Οι βασικές παράμετροι που καθορίζουν τη ρύθμιση στα πειράματα που παρέχετε είναι:

- **Μέγεθος πληθυσμού (Population size)** : 45 κανόνες
- **Μέγιστος αριθμός κανόνων στο σύστημα (Maximum Rules)** : 60
- **Μήκος συνθήκης (Condition length)** : 6 bits (που αντιστοιχεί στα έξι bits εισόδου)
- **Μήκος δράσης (Action length)** : 1 bit
- **Ποσοστό μεταλλαγής (Mutation Rate)** : 0,01
- **Ποσοστό προσφοράς (Bid percentage)** : 0,05 (το 5 % της ισχύος ενός κανόνα χρησιμοποιείται ως προσφορά)
- **Ποσό ανταμοιβής για σωστή πρόβλεψη (Reward Amount)** : 2,0
- **Αρνητικός συντελεστής ανταμοιβής για λανθασμένη πρόβλεψη (Negative reward)** : 1,25
- **Φορολογικός συντελεστής επί της ισχύος μετά από κάθε πέρασμα (Tax rate)** : 0,10 (10 % μείωση)
- **Όριο ισχύος για «κανόνες υψηλής ισχύος» (Strength threshold)** : 25

- Ποσοστό **ελιτισμού (Elitism)** στο **GeneticAlgorithm** : 0,25 (το 25% των κανόνων με την υψηλότερη ισχύ αντιγράφεται χωρίς αλλαγές στην επόμενη γενιά)
- Επαναλήψεις **Bucket Brigade** ανά γενιά (**Repeat count**) : 20 περάσματα πάνω από το ανακατεμένο σύνολο εκπαίδευσης

Αυτές οι παράμετροι εκτυπώνονται στην κορυφή κάθε αρχείου αποτελεσμάτων και στα αντιπροσωπευτικά πειράματα παραμένουν σταθερές.

Η κύρια πειραματική παρέμβαση είναι ο μέγιστος αριθμός γενεών, ο οποίος ελέγχει τη διάρκεια της εξέλιξης:

- Σύντομη εκπαίδευση: 2 000 γενεές
- Μεσαία εκπαίδευση: 6 000 γενεές
- Μακρά εκπαίδευση: 14 000 γενεές

Σε κάθε πείραμα, το πρόγραμμα εκτελεί αρκετές ανεξάρτητες δοκιμές (στον κώδικά σας: έως 10 ανά διαμόρφωση). Για κάθε δοκιμή, το πρόγραμμα:

- ▶ Αρχικοποιεί έναν νέο τυχαίο πληθυσμό κανόνων.
- ▶ Εφαρμόζει επαναληπτικά το Bucket Brigade και το GA για τον καθορισμένο αριθμό γενεών.

- ▶ Στο τέλος, αξιολογεί το τελικό σύνολο κανόνων σε ολόκληρο το σύνολο εκπαίδευσης και υπολογίζει την τελική ακρίβεια (ποσοστό σωστά ταξινομημένων προτύπων).
- ▶ Καταγράφει:
 - ▶ Τους «Τελικούς κανόνες υψηλής ισχύος» (κανόνες με ισχύ > όριο),
 - ▶ Τον πλήρη κατάλογο κανόνων (συνθήκη, ενέργεια, ισχύς),
 - ▶ Την τελική ακρίβεια για την συγκεκριμένη εκτέλεση.

Στο τέλος του πειράματος, ο κώδικας υπολογίζει και καταγράφει επίσης τη μέση ακρίβεια και τη μέγιστη ακρίβεια για όλες τις εκτελέσεις της συγκεκριμένης διαμόρφωσης.

7.2 Αποτελέσματα για σύντομες εξελικτικές εκτελέσεις (2 000 γενιές).

Στη διαμόρφωση με **MaxGenerations** = 2 000, το **CS** έχει σχετικά περιορισμένο χρόνο για να βελτιώσει και να εξελίξει το σύνολο των κανόνων του. Το αντιπροσωπευτικό αρχείο αποτελεσμάτων με αυτή τη ρύθμιση δείχνει ότι οι τελικές ακρίβειες σε όλες τις εκτελέσεις είναι μέτριες:

- Παράδειγμα εκτέλεσης: 56,25 % σωστές
- Παράδειγμα εκτέλεσης: 67,19 % σωστές

Αυτές οι ακρίβειες αντιστοιχούν στη σωστή **ταξινόμηση** περίπου 36-43 από τα **64 μοτίβα** εισόδου. Αυτό δείχνει ότι ακόμη και με μέτριο χρόνο

εξέλιξης, το σύστημα είναι σε θέση να ανακαλύψει ένα μη αμελητέο τμήμα της λογικής του **πολυπλέκτη**, αλλά η βάση κανόνων που έχει μάθει είναι ακόμα ατελής και υποβέλτιστη.

Οι κανόνες υψηλής αντοχής παρουσιάζουν ήδη μια σημαντική δομή. Τυπικά παραδείγματα περιλαμβάνουν:

000### → 0 και 001### → 1
01#0## → 0 και 01#1## → 1
10##0# → 0 και 10##1# → 1
11###0 → 0 και 11###1 → 1

Αυτά τα μοτίβα είναι συμβατά με τις τέσσερις περιπτώσεις επιλογής:

- **00** (πρώτα δύο bit **00**): η έξοδος εξαρτάται από το τρίτο bit **D0**,
- **01**: η έξοδος εξαρτάται από το τέταρτο bit **D1**,
- **10**: η έξοδος εξαρτάται από το πέμπτο bit **D2**,
- **11**: η έξοδος εξαρτάται από το έκτο bit **D3**.

Ωστόσο, στις **2.000** γενιές, αυτοί οι καλοί κανόνες συνυπάρχουν με πολλούς πιο αδύναμους ή εν μέρει αντικρουόμενους κανόνες. Η **γενετική αναζήτηση** δεν είχε ακόμη αρκετό χρόνο για να εξαλείψει τους περισσότερους άχρηστους κανόνες και να ενισχύσει ένα σχεδόν ελάχιστο, συνεπές **σύνολο κανόνων**. Ο **πληθυσμός** που προκύπτει παράγει επομένως σωστές προβλέψεις για ένα σημαντικό μέρος των εισροών, αλλά εξακολουθεί να κάνει λάθη σε αρκετές περιοχές του χώρου εισροών.

7.3 Αποτελέσματα για μεσαίες εξελικτικές εκτελέσεις (6 000 γενιές).

Όταν ο αριθμός των γενεών αυξάνεται σε 6 000, με όλες τις άλλες παραμέτρους να παραμένουν αμετάβλητες, η ποιότητα των εξελιγμένων πληθυσμών κανόνων βελτιώνεται σαφώς. Το αντιπροσωπευτικό αρχείο αποτελεσμάτων για MaxGenerations = 6 000 περιέχει εκτελέσεις με σημαντικά υψηλότερη ακρίβεια, για παράδειγμα:

- 78,13 % τελική ακρίβεια
- 81,25 % τελική ακρίβεια

Εδώ το σύστημα ταξινομεί σωστά περίπου 50-52 από τα 64 μοτίβα. Αυτή η αύξηση σε σύγκριση με την περίπτωση των 2 000 γενεών δείχνει ότι ο συνδυασμός **Bucket Brigade** και **GA** επωφελείται σημαντικά από τον παρατεταμένο εξελικτικό χρόνο: τα καλά σχήματα ενισχύονται και αναπαράγονται, ενώ οι κακοί κανόνες αποδυναμώνονται προοδευτικά και αντικαθίστανται.

Τα σύνολα κανόνων υψηλής ισχύος σε αυτές τις εκτελέσεις είναι πιο συμπαγή και ευθυγραμμίζονται πιο σαφώς με την επιδιωκόμενη λογική του πολυπλέκτη. Συνήθως περιλαμβάνουν κανόνες όπως:

- Επιλογή **00**:
 - ♦ **000### → 0**
 - ♦ **001### → 1**
- Επιλογή **01**:
 - ♦ **01#0## → 0**
 - ♦ **01#1## → 1**

- Επιλογή **10**:
 - ♦ **10##0#** → **0**
 - ♦ **10##1#** → **1**
- Επιλογή **11**:
 - ♦ **11###0** → **0**
 - ♦ **11###1** → **1**

Αυτοί οι κανόνες εμφανίζονται συχνά με δυνάμεις κοντά στις τιμές κορεσμού που παρατηρήθηκαν στα πειράματα (περίπου 30, ξεκινώντας από μια αρχική δύναμη 10). Η συχνή επανάληψη αυτών των σχημάτων με μικρές παραλλαγές στις θέσεις των **συμβόλων #** δείχνει πώς ο **ΓΑ** διαδίδει επιτυχημένα μοτίβα σε ολόκληρο τον **πληθυσμό**.

Ταυτόχρονα, ο **πληθυσμός** εξακολουθεί να περιέχει ορισμένους κανόνες εξερεύνησης μέσης δύναμης. Αυτοί συμβάλλουν στη συνεχή αναζήτηση και μπορούν περιστασιακά να διορθώσουν μικρά κενά στην κάλυψη, αλλά η διαδικασία λήψης αποφάσεων κυριαρχείται από τους ισχυρότερους, πιο συχνά ενισχυμένους κανόνες.

Συνολικά, τα πειράματα **6 000 γενεών** απεικονίζουν μια σαφή μετάβαση από τη μερική μάθηση σε μια ισχυρή, αν και όχι ακόμη τέλεια, απόδοση. Το σύστημα καταγράφει το μεγαλύτερο μέρος της λογικής δομής του πολυπλέκτη, αλλά εξακολουθεί να αφήνει ένα υποσύνολο προτύπων που έχουν ταξινομηθεί εσφαλμένα.

7.4 Αποτελέσματα για μακρές εξελικτικές εκτελέσεις (14 000 γενιές).

Η διαμόρφωση με **MaxGenerations** = 14 000 δίνει στο σύστημα αρκετό χρόνο για να βελτιώσει την εσωτερική βάση κανόνων του. Στο αντίστοιχο αρχείο αποτελεσμάτων, η καλύτερη παρατηρούμενη εκτέλεση επιτυγχάνει **τελική ακρίβεια 93,75 %**, πράγμα που σημαίνει ότι 60 από τα 64 μοτίβα ταξινομούνται σωστά. Άλλες εκτελέσεις στην ίδια διαμόρφωση εξακολουθούν να παρουσιάζουν κάποια μεταβλητότητα, με ακρίβεια που κυμαίνεται από **70% έως 80%**, αλλά η παρουσία τουλάχιστον μίας εκτέλεσης με πολύ υψηλή απόδοση αποτελεί ισχυρή απόδειξη της ικανότητας του **CS** να προσεγγίζει πολύ στενά τη λειτουργία του **πολυπλέκτη**.

Οι κανόνες υψηλής ισχύος σε αυτές τις μακρές εκτελέσεις είναι εντυπωσιακά καθαροί και ερμηνεύσιμοι. Συνήθως περιλαμβάνουν ένα μικρό σύνολο κυρίαρχων σχημάτων που κωδικοποιούν σχεδόν άμεσα τον πίνακα αληθείας του **πολυπλέκτη**, για παράδειγμα:

- **000### → 0, 001### → 1 (selector 00, D₀)**
- **01#0## → 0, 01#1## → 1 (selector 01, D₁)**
- **10##0# → 0, 10##1# → 1 (selector 10, D₂)**
- **11###0 → 0, 11###1 → 1 (selector 11, D₃)**

Αυτοί οι κανόνες συχνά συνοδεύονται από μερικές επιπλέον, ελαφρώς πιο συγκεκριμένες ή πιο γενικές παραλλαγές (για παράδειγμα με περισσότερα ή λιγότερα σύμβολα #), αλλά όλοι υποστηρίζουν μια

συνεπή κατάτμηση του χώρου εισόδου με βάση τα bit επιλογής και το αντίστοιχο bit δεδομένων.

Σε αυτό το στάδιο, πολλοί από τους προηγούμενους διερευνητικούς ή θορυβώδεις κανόνες είτε:

- έχουν αποδυναμωθεί λόγω λανθασμένων προβλέψεων και του μηχανισμού φόρου, είτε
- έχουν αφαιρεθεί και αντικατασταθεί κατά τη διάρκεια των επαναλήψεων **GA**.

Ο **πληθυσμός** κυριαρχείται επομένως από έναν μικρό πυρήνα κανόνων υψηλής καταλληλότητας που εφαρμόζουν συλλογικά σχεδόν το σύνολο της συμπεριφοράς του πολυπλέκτη. Οι υπόλοιπες λανθασμένες ταξινομήσεις (στην εκτέλεση με **ακρίβεια 93,75 %**) οφείλονται πιθανώς σε έναν μικρό αριθμό οριακών προτύπων όπου ο ισχυρότερος κανόνας δεν ταιριάζει ακριβώς με την επιθυμητή αντιστοίχιση ή όπου η ισοπαλία μεταξύ εξίσου ισχυρών κανόνων ευνοεί περιστασιακά μια ελαφρώς λανθασμένη ενέργεια (**Action**).

7.5 Παγκόσμιες τάσεις και ποιοτικές παρατηρήσεις.

Παρόλο που κάθε εκτέλεση είναι στοχαστική και τα μεμονωμένα αρχεία αποτελεσμάτων μπορεί να διαφέρουν, η συνολική εικόνα που προκύπτει από τη συλλογή των πειραμάτων είναι συνεπής:

- **Επίδραση του εξελικτικού χρόνου**

- Με **2 000** γενιές, το **σύστημα** μαθαίνει ένα σημαντικό μέρος της εργασίας, αλλά παραμένει μακριά από το τέλειο.
- Με **6.000** γενιές, ανακαλύπτει αξιόπιστα την κύρια λογική δομή και επιτυγχάνει ακρίβεια περίπου **78-81%**.
- Με **14.000** γενιές, είναι σε θέση, τουλάχιστον σε ορισμένες εκτελέσεις, να επιτύχει πολύ υψηλή ακρίβεια κοντά στο **94%**, υποδηλώνοντας σχεδόν πλήρη γνώση της χαρτογράφησης.
- **Εμφάνιση γενικευμένων κανόνων**
Σε όλες τις διαμορφώσεις, οι ισχυρότεροι κανόνες τείνουν να εκμεταλλεύονται το σύμβολο # για να γενικεύουν πολλαπλές εισόδους. Αυτό είναι κρίσιμο για το πρόβλημα του πολυπλέκτη: αντί να αποθηκεύει 64 μεμονωμένα μοτίβα, το CS συγκλίνει προς ένα μικρό σύνολο γενικευμένων κανόνων (π.χ. 000###, 11###1) που καλύπτουν ο καθένας μια ολόκληρη λογική περίπτωση. Αυτό αντιπροσωπεύει πραγματική δομική μάθηση και όχι απλή απομνημόνευση.
- **Πλεονασμός και σταθερότητα**
Τα σχήματα υψηλής αντοχής εμφανίζονται συχνά πολλές φορές στους τελικούς πληθυσμούς. Αυτός ο πλεονασμός αυξάνει την ανθεκτικότητα: ακόμη και αν ένα παράδειγμα μεταλλαχθεί ελαφρώς στις επόμενες γενιές, τα άλλα παραμένουν για να διατηρήσουν τη συμπεριφορά που έχει αποκτηθεί. Αντικατοπτρίζει επίσης την τάση του **GA** να αναπαράγει ευρέως τα κατάλληλα μοτίβα μέσω της επιλογής και της διασταύρωσης.
- **Μεταβλητότητα από εκτέλεση σε εκτέλεση**
Επειδή η αρχικοποίηση, η επιλογή, η διασταύρωση και η μετάλλαξη είναι όλες στοχαστικές, διαφορετικές εκτελέσεις με τις

ίδιες ρυθμίσεις παραμέτρων μπορούν να αποδώσουν κάπως διαφορετικές τελικές ακρίβειες. Ορισμένες εκτελέσεις συγκλίνουν σε ισχυρότερα σύνολα κανόνων από άλλες. Ωστόσο, η τάση με την αύξηση των γενεών είναι μονοτονική: οι καλύτερες εκτελέσεις σε μακρύτερα πειράματα υπερτερούν σταθερά εκείνων σε βραχύτερα.

7.6 Περίληψη.

Η πειραματική διάταξη αποτελείται από επαναλαμβανόμενες εκτελέσεις του ίδιου **συστήματος ταξινόμησης** σε ένα πλήρες **σύνολο εκπαίδευσης πολυπλέκτη**, με **σταθερές παραμέτρους αλγορίθμου** και **μεταβλητό μήκος εξέλιξης**. Τα αποτελέσματα σε πολλά αρχεία κειμένου δείχνουν μια σαφή πρόοδο:

- Οι σύντομες εκτελέσεις παράγουν μερικές αλλά ατελείς λύσεις.
- Οι εκτελέσεις μέσου μήκους ανακαλύπτουν μια ισχυρή προσέγγιση της υποκείμενης λογικής.
- Οι εκτεταμένες εκτελέσεις είναι ικανές να εξελίξουν μια συμπαγή, εξαιρετικά κατάλληλη βάση κανόνων που αναπαράγει σχεδόν τέλεια τη συμπεριφορά του **πολυπλέκτη 4 προς 1**.

Οι κανόνες υψηλής ισχύος που εξάγονται στο τέλος των εκτελέσεων είναι ερμηνεύσιμοι και ευθυγραμμίζονται στενά με τη θεωρητική δομή του προβλήματος, αποδεικνύοντας ότι η συνδυασμένη αρχιτεκτονική **BucketBrigade+GA** είναι σε θέση να ανακαλύψει και να σταθεροποιήσει σημαντικούς κανόνες **if-then** αντί για αυθαίρετες αντιστοιχίσεις προτύπων.

Αυτή η συλλογή εκτελέσεων παρέχει επομένως ισχυρή εμπειρική

υποστήριξη για την αποτελεσματικότητα του εφαρμοσμένου **συστήματος ταξινόμησης** ως προσαρμοστικού, βασισμένου σε κανόνες μηχανισμού μάθησης για δομημένες εργασίες ταξινόμησης, όπως ο **πολυπλέκτης**.

8 Περιγραφή της δεύτερης εφαρμογής των κανόνων που αναπτύχθηκαν από τον ΓΑ, για την εξαγωγή συμπερασμάτων.

Αφού το **Σύστημα Ταξινόμησης (CS)** ολοκληρώσει την εξελικτική φάση εκπαίδευσης και παράγει ένα τελικό σύνολο κανόνων μέσω του **Γενετικού Αλγορίθμου (ΓΑ)** και του μηχανισμού **Bucket Brigade**, χρησιμοποιείται ένα δεύτερο στάδιο εφαρμογής. Σε αυτό το στάδιο, οι εξελιγμένοι κανόνες δεν τροποποιούνται πλέον ούτε ανταμείβονται. Αντίθετα, αντιμετωπίζονται ως μια σταθερή βάση γνώσεων και χρησιμοποιούνται για την εξαγωγή συμπερασμάτων, την ανάλυση της συμπεριφοράς τους και την αξιολόγηση της αξιοπιστίας τους στο υποκείμενο πρόβλημα του **πολυπλέκτη**.

Ο σκοπός αυτής της δεύτερης εφαρμογής είναι διπλός:

- Να χρησιμοποιηθεί το τελικό σύνολο κανόνων ως μηχανισμός συμπερασμού για τον **πολυπλέκτη 4 προς 1**, δοκιμάζοντας συστηματικά τη συμπεριφορά του σε όλες τις πιθανές εισόδους.
- Να αξιολογήσει και να ερμηνεύσει μεμονωμένους κανόνες, μετρώντας πόσο συχνά κάθε κανόνας συμβάλλει σωστά ή λανθασμένα και προσδιορίζοντας τους πιο αξιόπιστους και

γενικούς κανόνες που ανακάλυψε το **GA**.

Το αντίστοιχο πρόγραμμα C++ υλοποιεί αυτήν την ανάλυση ως αυτόνομο εργαλείο που διαβάζει τους κανόνες από τα αρχεία κειμένου που παράχθηκαν στην πρώτη φάση και τους εφαρμόζει σε όλους τους 64 πιθανούς συνδυασμούς εισόδων 6 bit.

8.1 Εισαγωγή: φόρτωση της εξελιγμένης βάσης κανόνων από την έξοδο **GA**.

Το δεύτερο πρόγραμμα δεν εκτελεί το **GA** ή το **Bucket Brigade**. Αντ' αυτού, εισάγει την τελική κατάσταση του συστήματος ταξινόμησης από ένα αρχείο. Αυτό το αρχείο είναι ένα από τα αρχεία κειμένου «final_results_...» που παράγονται από το πρόγραμμα εκπαίδευσης.

Για να γίνει αυτή η διαδικασία φιλική προς τον χρήστη, ο κώδικας χρησιμοποιεί ένα παράθυρο διαλόγου επιλογής αρχείων των Windows (**GetOpenFileNameW**) μέσω της συνάρτησης **selectFile()**. Ο χρήστης καλείται να επιλέξει ένα αρχείο αποτελεσμάτων και, στη συνέχεια, το πρόγραμμα το ανοίγει και αναζητά την ενότητα με την ένδειξη:

ALL RULES

Από εκείνο το σημείο και μετά, κάθε γραμμή που ταιριάζει με το μοτίβο:

Κανόνας k: Συνθήκη = <συνθήκη>, Ενέργεια = <a>, Δύναμη = <s>

αναλύεται χρησιμοποιώντας μια κανονική έκφραση στη συνάρτηση **parseAllRulesLine**. Για κάθε τέτοια γραμμή, μια δομή **Rule**

συμπληρώνεται με:

- **condition** – το δυαδικό μοτίβο με πιθανά σύμβολα #;
- **action** – το προβλεπόμενο αποτέλεσμα του κανόνα (0 ή 1);
- **strength** – η τελική ισχύς που αποδίδεται κατά τη διάρκεια της εκπαίδευσης;
- **success and failures** – μετρητές, αρχικά ρυθμισμένοι στο μηδέν, που θα συμπληρωθούν κατά τη διάρκεια της φάσης αξιολόγησης.

Αυτό σημαίνει ότι το ακριβές τελικό σύνολο κανόνων που παράγεται από το **GA** και το **Bucket Brigade** στην πρώτη φάση εισάγεται και επαναχρησιμοποιείται ως στατική βάση κανόνων για τη δεύτερη εφαρμογή.

8.2 Δημιουργία όλων των πιθανών εισόδων και εξόδων αναφοράς.

Η δεύτερη εφαρμογή αξιολογεί τους κανόνες όχι μόνο με βάση τα παραδείγματα εκπαίδευσης που εμφανίστηκαν σε συγκεκριμένη σειρά κατά τη διάρκεια της πρώτης φάσης, αλλά και συστηματικά με βάση και τις **64** πιθανές εισόδους **6 bit**.

Για το σκοπό αυτό, το πρόγραμμα επαναλαμβάνει όλους τους ακέραιους αριθμούς από το 0 έως το 63 και μετατρέπει κάθε αριθμό σε μια δυαδική συμβολοσειρά 6 bit:

```
for (int i = 0; i < 64; ++i) {  
    std::string data;  
    for (int b = 5; b >= 0; --b)
```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```
data += ((i >> b) & 1) ? '1' : '0';  
  
....  
}
```

Για κάθε τέτοια σειρά δεδομένων, η πραγματική έξοδος του πολυπλέκτη υπολογίζεται από τη συνάρτηση `multiplexer6bit`:

- Τα δύο πρώτα bit (**data[0]** και **data[1]**) ερμηνεύονται ως διεύθυνση (επιλογέας) **S1S0**, που μετατρέπεται σε ακέραιο αριθμό στην περιοχή 0–3.
- Η έξοδος είναι τότε **data[2 + addr]**, η οποία αντιστοιχεί σε ένα από τα τέσσερα bit δεδομένων (**D0, D1, D2, D3**).

Αυτό παρέχει, για κάθε πιθανή είσοδο, μια ετικέτα πραγματικής κατάστασης με την οποία μπορούν να συγκριθούν οι προβλέψεις που βασίζονται σε κανόνες.

8.3 Εφαρμογή των κανόνων: ενεργοποίηση, αντιστοίχιση και επιλογή νικητή.

Για κάθε ένα από τα **64 μοτίβα εισόδου**, το πρόγραμμα καθορίζει ποιοι από τους εισαγόμενους κανόνες θα ενεργοποιηθούν από την είσοδο αυτή και πώς θα συμπεριφερθούν. Αυτό υλοποιείται μέσω της συνάρτησης αντιστοίχισης:

- Η συνθήκη ενός κανόνα αντιστοιχεί σε μια είσοδο εάν, για κάθε θέση *i*, είτε:
 - το bit της συνθήκης είναι ίσο με το bit της εισόδου, ή

- ο το bit της συνθήκης είναι «#» (δεν έχει σημασία), το οποίο ταιριάζει τόσο με το 0 όσο και με το 1.

Όλοι οι κανόνες των οποίων οι συνθήκες ταιριάζουν με την είσοδο σχηματίζουν το σύνολο αντιστοίχισης για αυτό το μοτίβο. Για κάθε κανόνα σε αυτό το σύνολο αντιστοίχισης:

- Εάν η **rule.action** ισούται με τη σωστή έξοδο του **πολυπλέκτη**, ο μετρητής **επιτυχιών(Success)** του κανόνα αυξάνεται.
- Διαφορετικά, ο μετρητής **αποτυχιών(failures)** του αυξάνεται.

Επιπλέον, το **πρόγραμμα** καθορίζει, για κάθε είσοδο, ποιος από τους κανόνες αντιστοίχισης θα έλεγχε πραγματικά την έξοδο εάν η βάση κανόνων χρησιμοποιούνταν ως μηχανή συμπερασμού. Αυτό γίνεται επιλέγοντας, μεταξύ όλων των κανόνων αντιστοίχισης, αυτόν με τη **μεγαλύτερη ισχύ(highest strength)** (όπως έχει μάθει το **GA**). Ο δείκτης αυτού του κανόνα αποθηκεύεται ως ο «**νικητής**» για την είσοδο αυτή. Συνοψίζοντας, για κάθε μοτίβο εισόδου, η δεύτερη εφαρμογή καταγράφει:

- ποιοι κανόνες ταιριάζουν με την είσοδο
- αν η ενέργεια κάθε κανόνα που ταιριάζει είναι σωστή ή λανθασμένη
- ποιος κανόνας θα κέρδιζε τον διαγωνισμό με βάση τη δύναμη.

Αυτό μας επιτρέπει να αναλύσουμε όχι μόνο τη συμπεριφορά των μεμονωμένων κανόνων, αλλά και τη δυναμική λήψης αποφάσεων ολόκληρου του **συστήματος ταξινόμησης**.

8.4 Λεπτομερής αναφορά ανά είσοδο.

Μετά την επεξεργασία και των **64** εισόδων, το πρόγραμμα εκτυπώνει, για κάθε μοτίβο εισόδου, μια λεπτομερή αναφορά εκτέλεσης. Για κάθε μία από τις **64** «εκτελέσεις» (σε αυτό το πλαίσιο, κάθε εκτέλεση αντιστοιχεί σε ένα ξεχωριστό μοτίβο εισόδου):

- Εμφανίζονται τα δυαδικά δεδομένα εισόδου.
- Παρατίθενται όλοι οι κανόνες που ταιριάζουν με την είσοδο, δείχνοντας:
 - την κατάστασή τους,
 - τη δράση τους,
 - την ισχύ τους,
 - τον αριθμό των επιτυχιών και των αποτυχιών που συσσωρεύτηκαν και στις 64 εισόδους.
- Ο κανόνας που προσδιορίστηκε ως νικητής για την εν λόγω είσοδο (υψηλότερη ισχύς μεταξύ των κανόνων αντιστοίχισης) επισημαίνεται με πράσινο χρώμα στην κονσόλα.

Αυτή η απεικόνιση ανά είσοδο επιτρέπει στον χρήστη να ελέγξει οπτικά πώς συμπεριφέρεται η εξελιγμένη βάση κανόνων για κάθε πιθανό συνδυασμό bit επιλογής και bit δεδομένων, και ποιοι κανόνες κυριαρχούν αποτελεσματικά στη διαδικασία λήψης αποφάσεων.

8.5 Ανάλυση απόδοσης παγκόσμιων κανόνων.

Αφού εκτυπωθούν οι αναφορές ανά είσοδο, το πρόγραμμα εκτελεί μια συγκέντρωση δεύτερου επιπέδου, αυτή τη φορά πάνω στους κανόνες και όχι στις εισόδους. Ταξινομεί ξανά τους κανόνες (εδώ κυρίως κατά ισχύ) και εκτυπώνει μια Αναφορά Απόδοσης Κανόνων, που δείχνει για κάθε

κανόνα:

- την κατάσταση του,
- τη δράση του,
- την τελική ισχύ του,
- τον συνολικό αριθμό επιτυχιών,
- τον συνολικό αριθμό αποτυχιών.

Οι κανόνες που έχουν τουλάχιστον **μία επιτυχία** και **μηδέν αποτυχίες** επισημαίνονται με μπλε χρώμα, χαρακτηρίζοντάς τους ως άψογους κανόνες σε σχέση με τον πλήρη **πίνακα αλήθειας 64 προτύπων**. Αυτοί οι κανόνες αντιπροσωπεύουν τα πιο αξιόπιστα και γενικά στοιχεία λήψης αποφάσεων που ανακάλυψε το **GA**. Δεν κάνουν ποτέ λανθασμένες προβλέψεις για καμία είσοδο με την οποία ταιριάζουν.

Αυτή η τελική αναφορά είναι ιδιαίτερα σημαντική για την εξαγωγή συμπερασμάτων, επειδή επιτρέπει στον χρήστη:

- Να προσδιορίσει ποιοι κανόνες αντιπροσωπεύουν καθαρά, σωστά σχήματα της λογικής του **πολυπλέκτη** (για παράδειγμα, συνθήκες όπως **11###1** → **1** ή **01#0##** → **0** που δεν αποτυγχάνουν ποτέ).
- Να εντοπίσει κανόνες που είναι υπερβολικά γενικοί ή εν μέρει λανθασμένοι, όπως υποδεικνύεται από τον αριθμό αποτυχιών διαφορετικό από το μηδέν.
- Να παρατηρήσει τη σχέση μεταξύ της ισχύος (όπως διαμορφώνεται από την εξελικτική μάθηση) και της πραγματικής εμπειρικής αξιοπιστίας (όπως μετράται από την επιτυχία/αποτυχία σε ολόκληρο το χώρο εισόδου).

8.6 Ο ρόλος αυτής της δεύτερης εφαρμογής για την εξαγωγή συμπερασμάτων.

Από μεθοδολογική άποψη, αυτή η δεύτερη εφαρμογή των κανόνων χρησιμεύει ως επίπεδο αξιολόγησης και ερμηνείας μετά την εκπαίδευση. Ενώ το πρώτο πρόγραμμα επικεντρώνεται στην εκμάθηση και την εξέλιξη των κανόνων, αυτό το πρόγραμμα επικεντρώνεται στα εξής:

- **Χρήση των κανόνων** για την ταξινόμηση όλων των πιθανών εισροών ,
- **Μέτρηση της συμπεριφοράς** κάθε κανόνα σε ολόκληρο το χώρο του προβλήματος,
- **Την επισήμανση ισχυρών**, άψογων κανόνων που αποτυπώνουν τη βασική λογική του **πολυπλέκτη**,
- **Την αποκάλυψη ασθενέστερων κανόνων** που μπορεί να έχουν κάποια **ισχύ(Strength)**, αλλά δεν είναι πάντα σωστοί.

Ως αποτέλεσμα, αυτή η δεύτερη εφαρμογή επιτρέπει να εξαχθούν συγκεκριμένα συμπεράσματα σχετικά με:

- Την **ποιότητα** της γνώσης που ανακαλύφθηκε από το σύστημα ταξινόμησης,
- Την **ερμηνευσιμότητα** των εξελιγμένων κανόνων,
- Τον βαθμό **γενίκευσης** και **ορθότητας** που επιτεύχθηκε από τον πληθυσμό κανόνων που εκπαιδεύτηκε με **GA**.

9. Σχολιασμός των αποτελεσμάτων, της επίδρασης των παραμέτρων στα αποτελέσματα και ειδικότερα του συνόλου των κανόνων που αναπτύχθηκε από τη ΓΑ σε περιπτώσεις με 100% ακρίβεια.

9.1 Γενική επισκόπηση των αποτελεσμάτων σε όλες τις δοκιμές.

Σε όλα τα πειράματα που διεξήχθησαν με το **Σύστημα Ταξινόμησης (CS)**, παρατηρήθηκε ευρεία κατανομή των τελικών ακρίβειων. Πολλές δοκιμές συγκλίνουν σε μέτρια ή υψηλά επίπεδα ακρίβειας (**50-80%**), ενώ μόνο ένα υποσύνολο πέτυχε πλήρως σωστή απόδοση σε ολόκληρο τον **πίνακα αληθείας** του **πολυπλέκτη 64** προτύπων.

Η παρατηρούμενη μεταβλητότητα είναι αναμενόμενη, δεδομένης της στοχαστικής εξελικτικής φύσης του **GA** και των πολύπλοκων αλληλεπιδράσεων μεταξύ της ισχύος των κανόνων, της ενίσχυσης της αλυσίδας και της ανακάλυψης κανόνων.

Ωστόσο, το **σύστημα** απέδειξε την ικανότητά του να εξελίσσει σύνολα κανόνων ικανά να επιτύχουν **100% ακρίβεια ταξινόμησης**, επιβεβαιώνοντας ότι:

- Η αναπαράσταση του κανόνα είναι αρκετά εκφραστική ώστε να κωδικοποιήσει τη λειτουργία του **πολυπλέκτη 4 προς 1**.
- Η εξελικτική διαδικασία μπορεί, υπό κατάλληλες συνθήκες, να ανακαλύψει πλήρη και σωστά σύνολα σχημάτων.

- Ο συνδυασμός **GA + Bucket Brigade** αποτελεί έναν αποτελεσματικό μηχανισμό μάθησης για αυτό το είδος προβλήματος **ταξινόμησης Boolean**.

Η ύπαρξη πλήρως ακριβών εκτελέσεων αποδεικνύει ότι οι θεωρητικές προσδοκίες των συστημάτων ταξινόμησης τύπου **Holland** ικανοποιούνται στην πράξη.

9.2 Επίδραση των ρυθμίσεων παραμέτρων στα αποτελέσματα.

Μέσω της σύγκρισης των εκτελέσεων υψηλής και χαμηλής ακρίβειας, προκύπτουν διάφορες σημαντικές παρατηρήσεις σχετικά με την επίδραση των παραμέτρων.

9.2.1 Ποσοστό μετάλλαξης (0,01).

Ένα χαμηλό ποσοστό μετάλλαξης εξασφάλισε ότι οι δομές των κανόνων δεν διαταράσσονταν συνεχώς, επιτρέποντας στα ευνοϊκά σχήματα να παραμείνουν και να βελτιωθούν.

Οι εκτελέσεις με **ακρίβεια 100%** έδειξαν ότι αυτός ο ρυθμός ήταν επαρκής για:

- την εισαγωγή νέων χρήσιμων γενικεύσεων (συνθήκες βασισμένες στο «#»),
- την επιδιόρθωση υποβέλτιστων κανόνων,
- την αποφυγή πρόωρης σύγκλισης,
- διατηρώντας παράλληλα τους βασικούς ακριβείς κανόνες μόλις εμφανίζονταν.

Η υπερβολική μετάλλαξη θα είχε καταστρέψει αυτά τα μοτίβα πριν φτάσουν σε κυριαρχική θέση.

9.2.2 Μέγεθος πληθυσμού (45) και μέγιστος αριθμός κανόνων (60).

Ο πολυπλέκτης 4 προς 1 απαιτεί σχετικά μικρό αριθμό κανόνων για να εκφραστεί σωστά. Ένας **πληθυσμός 45-60 κανόνων** επιτρέπει την **ποικιλομορφία**, αλλά παραμένει υπολογιστικά διαχειρίσιμος.

Η ανάλυση δείχνει:

- Οι εκτελέσεις **χαμηλότερης ακρίβειας** συχνά είχαν ανεπαρκή ποικιλομορφία στην αρχή.
- Οι εκτελέσεις **υψηλής ακρίβειας** διατήρησαν πολλαπλά ανταγωνιστικά σύμπλεγματα κανόνων, αυξάνοντας τις εξελικτικές πιθανότητες ανακάλυψης των σωστών σχημάτων.

9.2.3 Ποσοστό προσφοράς (0,05) και σύστημα ανταμοιβής.

Ο μηχανισμός **Bucket Brigade** διαδραμάτισε σημαντικό ρόλο στη διαμόρφωση της ισχύος των κανόνων:

- Οι **σωστοί κανόνες** συγκέντρωναν ανταμοιβές ταχύτερα από τους λανθασμένους.
- Το **ποσοστό προσφοράς (Bid percentage)** έλεγχε το πόσο έντονα κυμαινόταν η ισχύς των κανόνων.
- Ένα υπερβολικά υψηλό ποσοστό προσφοράς προκαλεί αστάθεια, αλλά το 0,05 παρείχε μια «ήπια πίεση» κατάλληλη για δομημένες εργασίες.

Οι εκτελέσεις **υψηλής ακρίβειας** έδειξαν σταθερά μια ισχυρή διαφορά μεταξύ των επιτυχημένων και των ανεπιτυχών κανόνων.

9.2.4 Ελιτισμός (Elitism) (0,25).

Ο βαθμός ελιτισμού εξασφάλιζε ότι το 25% των καλύτερων κανόνων διατηρούνταν πάντα κατά την αναπαραγωγή. Αυτό ήταν απαραίτητο για:

- τη διατήρηση τέλειων ή σχεδόν τέλειων κανόνων,
- την αποτροπή της γενετικής μετατόπισης από τη διαγραφή καλών λύσεων,
- τη σταθεροποίηση της εξέλιξης στις επόμενες γενιές.

Οι εκτελέσεις που πέτυχαν 100% ακρίβεια επωφελήθηκαν σαφώς από τον ελιτισμό, ο οποίος επέτρεψε στο σύστημα να «κλειδώσει» τα σωστά μερικά σχήματα μόλις ανακαλύφθηκαν.

9.2.5 Παράγοντας αρνητικής ανταμοιβής (Negative Reward Factor) (1,25).

Η **τιμωρία** των λανθασμένων ενεργειών ήταν κρίσιμη για την εξάλειψη των παραπλανητικών κανόνων.

Οι **τέλειες εκτελέσεις** παρουσίαζαν σταθερά:

- λιγότερους αδύναμους αλλά λανθασμένους κανόνες,
- ισχυρότερο διαχωρισμό μεταξύ κανόνων υψηλής και χαμηλής καταλληλότητας,

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

- ταχεία εξαφάνιση θορυβωδών σχημάτων.

Αυτό αποδεικνύει ότι η αρνητική ενίσχυση οδηγεί τον πληθυσμό των κανόνων προς την ακρίβεια.

9.3 Λεπτομερής σχολιασμός για τις εκτελέσεις με 100% ακρίβεια.

Δύο ανεξάρτητες εκτελέσεις πέτυχαν 100% ακρίβεια:

- run 4 στο αρχείο final_results_2024-06-09_11-21-03.txt
- run 9 στο αρχείο final_results_2024-06-09_12-09-42.txt

Αν και οι ακριβείς κανόνες δεν μπορούν να εξαχθούν από τις περικομμένες προεπισκοπήσεις, γνωρίζουμε τα εξής:

9.3.1 Τα τέλεια σύνολα κανόνων περιέχουν πάντα καθαρά σχήματα που γενικεύονται σωστά.

Για έναν πολυπλέκτη 4 προς 1, το ιδανικό σύνολο κανόνων περιέχει συνήθως κανόνες όπως:

00#### → D0

01#### → D1

10#### → D2

11#### → D3

Η πιο συγκεκριμένες υλοποιήσεις:

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

00#0## → 0

00#1## → 1

01##0# → 0

01##1# → 1

κ.λπ.

Οι τέλειες εκτελέσεις χαρακτηρίζονται από:

- Κανένας αντιφατικός κανόνας δεν παραμένει στον **πληθυσμό**.
- Σπάνια χρήση των **συμβόλων «#»**, αλλά ακριβώς όση χρειάζεται για τη σωστή γενίκευση.
- **Σαφείς κανόνες επιλογής προτύπων (S1S0)** που κυριαρχούν στον πληθυσμό.
- **Τιμές ισχύος(Strength)** που σχηματίζουν μια απότομη «κορυφή καταλληλότητας» γύρω από άψογους κανόνες.

Αυτά τα σύνολα κανόνων αντικατοπτρίζουν την υποκείμενη δομή **Boolean** του πολυπλέκτη και αποδεικνύουν ότι το **CS** έμαθε τη σωστή λειτουργική αποσύνθεση.

9.3.2 Κυριαρχία των κανόνων που καθορίζονται από τον επιλογή.

Σε κάθε **πολυπλέκτη**, τα δύο πρώτα **bits** καθορίζουν ποιο **bit** δεδομένων είναι σχετικό. Σε τέλειες εκτελέσεις:

- οι κανόνες που εξαρτώνται από τα **00, 01, 10** και **11** προέκυψαν φυσικά,
- συσώρευσαν συνεπή ανταμοιβή,
- υπερίσχυσαν των κανόνων που προσπαθούσαν να αναμίξουν επιλογείς με άσχετα bits δεδομένων.
- Έτσι, η εξέλιξη ανακάλυψε τη βασική δομή του πολυπλέκτη χωρίς να της δοθεί ρητά.

9.3.3 Απουσία υπερβολικά γενικευμένων ή υπογενικευμένων κανόνων.

Τα σύνολα κανόνων **υψηλής ακρίβειας** παρουσιάζουν:

- σχεδόν κανέναν κανόνα με πάρα πολλά **σύμβολα #** (υπερβολικά γενικευμένοι, επιρρεπείς σε σφάλματα)
- σχεδόν κανέναν υπερβολικά συγκεκριμένο κανόνα που να ταιριάζει μόνο με ένα μοτίβο (υπογενικευμένοι)

Αντ' αυτού, η **ΓΑ** συγκλίνει σε ένα μεσαίου επιπέδου αφηρημένο επίπεδο, σύμφωνα με τη θεωρία σχημάτων του **Holland**.

9.4 Γιατί ορισμένες εκτελέσεις φτάνουν το 100% και άλλες όχι.

Ακόμη και με πανομοιότυπους παραμέτρους, οι στοχαστικοί παράγοντες επηρεάζουν τη σύγκλιση:

9.4.1 Η έγκαιρη ανακάλυψη κανόνων έχει σημασία.

Εάν μια πρώιμη γενιά δημιουργήσει τυχαία έναν μερικώς σωστό κανόνα βασισμένο σε επιλογέα (π.χ. 00####), τείνει να:

- κερδίζει ανταμοιβή με συνέπεια,
- συσσωρεύει δύναμη,
- λειτουργεί ως άγκυρα γύρω από την οποία εξελίσσονται πιο τέλειοι απόγονοι.

Οι εκτελέσεις που δεν συναντούν ποτέ αυτά τα πολλά υποσχόμενα σχήματα νωρίς μπορεί να «κολλήσουν» σε υποβέλτιστες περιοχές του τοπίου καταλληλότητας.

9.4.2 Η γενετική ποικιλομορφία παίζει σημαντικό ρόλο.

Οι τέλειες εκτελέσεις διατηρούν υψηλότερη ποικιλομορφία νωρίς στην εξέλιξη.

Οι εκτελέσεις χαμηλότερης ακρίβειας συχνά παρουσιάζουν:

- πρόωρη σύγκλιση,
- πληθυσμούς που κυριαρχούνται από μέτριους κανόνες,
- έλλειψη δομικών στοιχείων που απαιτούνται για την κατασκευή πλήρων λογικών προτύπων.

9.4.3 Η ισορροπία μεταξύ εξερεύνησης και εκμετάλλευσης είναι κρίσιμη.

Οι επιλεγμένες παράμετροι (ειδικά ο ελιτισμός, ο ρυθμός μετάλλαξης και η αρνητική ανταμοιβή) επιτυγχάνουν μια ευαίσθητη ισορροπία.

Οι εκτελέσεις με **100% ακρίβεια** είναι εκείνες στις οποίες:

- η εξερεύνηση (μέσω μετάλλαξης/διασταύρωσης) βρίσκει τις σωστές δομές,
- η εκμετάλλευση (μέσω ενίσχυσης και ελιτισμού) τις ενισχύει.

9.5 Συνολική ερμηνεία.

Η εμφάνιση συνόλων κανόνων με **100% ακρίβεια** αποδεικνύει ότι:

- Η **αρχιτεκτονική του συστήματος ταξινόμησης** είναι αρκετά εκφραστική για να λύσει το πρόβλημα του πολυπλέκτη.
- Ο **γενετικός αλγόριθμος** ανακαλύπτει με αξιοπιστία τις δομημένες κανονικότητες στον χώρο εισόδου.
- Η **ενισχυτική μάθηση(Reinforcement learning)** μέσω του **αλγορίθμου bucket brigade** σταθεροποιεί τους σωστούς κανόνες.
- Οι **επιλογές παραμέτρων** επηρεάζουν σημαντικά τη σύγκλιση, την ποικιλομορφία και την ποιότητα της λύσης.

Το πιο σημαντικό είναι ότι τα τέλεια σύνολα κανόνων παρουσιάζουν:

- καθαρή, ερμηνεύσιμη λογική δομή,
- σταθερές και ισχυρές αντιστοιχίσεις ενεργειών,
- ορθότητα σε όλες τις πιθανές εισόδους,
- ισχυρή ευθυγράμμιση με τη θεωρητική αποσύνθεση του σχήματος.

Αυτό επιβεβαιώνει ότι το εξελιγμένο σύστημα ταξινόμησης έμαθε πραγματικές συμβολικές γενικεύσεις και δεν απομνημόνευσε απλώς τα δεδομένα εκπαίδευσης.

10. Συμπεράσματα και οφέλη από την παρούσα εργασία.

Η **ανάπτυξη** και η πειραματική αξιολόγηση του συστήματος ταξινόμησης που εφαρμόστηκε σε αυτή την εργασία αποδεικνύει την ικανότητα της εξελικτικής και ενισχυτικής μάθησης κανόνων να επιλύει δομημένα προβλήματα **ταξινόμησης Boolean**, όπως ο **πολυπλέκτης 4 προς 1**. Μέσω της συνεργασίας του **γενετικού αλγορίθμου** και του μηχανισμού **Bucket Brigade**, το σύστημα μπόρεσε να ανακαλύψει, να βελτιώσει και να σταθεροποιήσει σύνολα συμβολικών κανόνων **if-then** που περιγράφουν με ακρίβεια τις υποκείμενες λογικές σχέσεις του προβλήματος. Η ικανότητα πολλών εκτελέσεων να επιτύχουν **100% ακρίβεια** επιβεβαιώνει ότι η επιλεγμένη αναπαράσταση, οι μηχανισμοί μάθησης και η παραμετροποίηση ήταν επαρκείς για να επιτρέψουν στο **σύστημα ταξινόμησης** να γενικεύσει πλήρως τη συνάρτηση-στόχο.

Ένα από τα κύρια οφέλη αυτής της εργασίας είναι ότι απεικονίζει τον τρόπο με τον οποίο τα προσαρμοστικά μοντέλα που βασίζονται σε κανόνες μπορούν να εξάγουν αυτόνομα σημαντικά σχήματα από παραδείγματα ακατέργαστων **εισόδων-εξόδων** χωρίς προηγούμενη δομική γνώση. Οι **κανόνες** που προκύπτουν δεν είναι παράμετροι μαύρου κουτιού, αλλά ερμηνεύσιμες λογικές εκφράσεις, προσφέροντας διαφανή εικόνα του τρόπου με τον οποίο το σύστημα λαμβάνει αποφάσεις. Αυτή η ερμηνευσιμότητα είναι ιδιαίτερα

πολύτιμη σε τομείς όπου η κατανόηση της διαδικασίας συλλογιστικής είναι εξίσου σημαντική με την επίτευξη υψηλής ακρίβειας.

Επιπλέον, το έργο υπογραμμίζει την επίδραση διαφορετικών παραμέτρων εξέλιξης και ενίσχυσης στην απόδοση του συστήματος. Με την εκτέλεση **πολλαπλών δοκιμών** υπό ποικίλες συνθήκες, η μελέτη καταδεικνύει τις αντισταθμίσεις μεταξύ εξερεύνησης και εκμετάλλευσης, τη σημασία της διατήρησης **κανόνων υψηλής ποιότητας** μέσω του **ελιτισμού** και τον ρόλο των **μεταλλάξεων** και των ενημερώσεων **ισχύος** στη διατήρηση της χρήσιμης ποικιλομορφίας. Αυτές οι παρατηρήσεις παρέχουν πρακτικές οδηγίες για το σχεδιασμό και τη ρύθμιση **συστημάτων ταξινόμησης** για **μελλοντικές εφαρμογές**.

Συνολικά, η εργασία δείχνει ότι τα συστήματα ταξινόμησης είναι μια ευέλικτη και ισχυρή μεθοδολογία για την εκμάθηση διακριτών αντιστοιχίσεων **εισόδου-εξόδου**, προσφέροντας τόσο ισχυρή ικανότητα γενίκευσης όσο και σαφείς συμβολικές αναπαραστάσεις. Οι γνώσεις που αποκτήθηκαν από αυτήν την εφαρμογή μπορούν να χρησιμεύσουν ως βάση για την εφαρμογή παρόμοιων εξελικτικών προσεγγίσεων βασισμένων σε κανόνες σε πιο σύνθετα προβλήματα ταξινόμησης, συλλογιστικής ή ομαδοποίησης, και να ανοίξουν δρόμους για περαιτέρω βελτιώσεις σε προσαρμοστικά, ερμηνεύσιμα **συστήματα μηχανικής μάθησης**.

11. Προτάσεις για μελλοντική επέκταση.

Ενώ το **σύστημα ταξινόμησης** που έχει εφαρμοστεί επιδεικνύει με επιτυχία τους **θεμελιώδεις μηχανισμούς** της **εξελικτικής εκμάθησης κανόνων** και επιτυγχάνει **υψηλή ακρίβεια** στην εργασία του **πολυπλέκτη**, εξακολουθούν να υπάρχουν αρκετές υποσχόμενες κατευθύνσεις για τη **μελλοντική ανάπτυξη** και επέκταση του συστήματος. Αυτές οι επεκτάσεις θα εμπλουτίσουν τις **υπολογιστικές δυνατότητες** του **μοντέλου** και θα διευρύνουν την εφαρμοσιμότητά του σε πιο σύνθετα προβλήματα εκμάθησης.

11.1 Επέκταση σε προβλήματα πολυπλέκτη υψηλότερης διάστασης.

Ένα φυσικό επόμενο βήμα είναι η αξιολόγηση του **συστήματος** σε μεγαλύτερες λειτουργίες **πολυπλέκτη**, όπως **διαμορφώσεις 8 προς 1 ή 16 προς 1**. Αυτά τα προβλήματα περιλαμβάνουν περισσότερες εισόδους και υψηλότερο βαθμό λογικής πολυπλοκότητας, απαιτώντας μακρύτερες συνθήκες κανόνων και μεγαλύτερο χώρο αναζήτησης. Η δοκιμή του **συστήματος** υπό αυτές τις **συνθήκες** θα αποκαλύψει την επεκτασιμότητά του, τα **όρια** της **μαθησιακής του ικανότητας** και την πιθανή ανάγκη για βελτιωμένους γενετικούς τελεστές ή πιο εκφραστικές αναπαραστάσεις.

11.2 Ενσωμάτωση αλυσιδωτής σύνδεσης κανόνων και πολυβήματης συλλογιστικής.

Η τρέχουσα εφαρμογή επικεντρώνεται στην **ταξινόμηση** ενός βήματος, όπου κάθε είσοδος αντιστοιχίζεται άμεσα σε μια ενέργεια. Μελλοντικές εργασίες ενδέχεται να ενσωματώσουν την ενεργοποίηση αλυσίδας κανόνων, επιτρέποντας στους κανόνες να υποστηρίζουν ενδιάμεσες συμπεράσματα. Αυτό θα επιτρέψει στο σύστημα να μοντελοποιήσει πολυβάθμιες διαδικασίες συλλογιστικής και να αντιμετωπίσει **προβλήματα** όπου τα **συμπεράσματα** πρέπει να προκύπτουν από ακολουθίες **ενεργοποιήσεων κανόνων** και όχι από απλές άμεσες αντιστοιχίσεις.

11.3 Προσαρμοστικός έλεγχος παραμέτρων.

Πολλές παράμετροι του **συστήματος** όπως ο **ρυθμός μετάλλαξης**, το **ποσοστό προσφοράς**, οι **τιμές ανταμοιβής** και ο **ελιτισμός** είναι **σταθερές** καθ' όλη τη διάρκεια της **εκπαίδευσης**. Η εισαγωγή μηχανισμών για δυναμική ή αυτοπροσαρμοστική ρύθμιση των παραμέτρων θα μπορούσε να βελτιώσει την αποδοτικότητα και την ανθεκτικότητα. Για παράδειγμα, οι **ρυθμοί μετάλλαξης** θα μπορούσαν να αυξηθούν κατά την αρχική εξερεύνηση και να μειωθούν καθώς η εξέλιξη σταθεροποιείται, ή οι τιμές ανταμοιβής θα μπορούσαν να προσαρμοστούν με βάση τη δυσκολία συγκεκριμένων καταστάσεων στο χώρο του προβλήματος.

11.4 Βελτιωμένη διατήρηση της ποικιλομορφίας.

Σε ορισμένες εκτελέσεις, η πρόωρη σύγκλιση του πληθυσμού κανόνων μείωσε την εξερεύνηση και εμπόδισε την ανακάλυψη βέλτιστων σχημάτων. **Μελλοντικές εφαρμογές** θα μπορούσαν να ενσωματώσουν μεθόδους όπως η κοινή χρήση καταλληλότητας, η εξειδίκευση ή η αναζήτηση καινοτομίας, εξασφαλίζοντας τη διατήρηση της ποικιλομορφίας και αποτρέποντας τη στασιμότητα. Αυτές οι τεχνικές θα βοηθούσαν το σύστημα να αποφύγει τους υποβέλτιστους ελκυστές στο τοπίο αναζήτησης.

11.5 Εργαλεία απεικόνισης και διάγνωσης.

Η επέκταση του συστήματος με γραφικά ή στατιστικά στοιχεία θα βελτιώσει σημαντικά την ερμηνεία. Οι πιθανές εργαλειαθικές περιλαμβάνουν:

- Χάρτες κάλυψης κανόνων
- Εικόνες εξέλιξης της ισχύος κανόνα ανά γενιά,
- κύκλοι σύγκλισης,
- ή οπτικοποιητές διαδρομής αποφάσεων.

Τέτοια εργαλεία θα βοηθήσουν τόσο την ανάλυση της έρευνας όσο και τη χρήση της εκπαίδευσης, προσφέροντας βαθύτερη

κατανόηση των εσωτερικών δυναμικών του **συστήματος ταξινόμησης**.

11.6 Υβριδισμός με άλλες τεχνικές μάθησης.

Το **σύστημα** θα μπορούσε να βελτιωθεί με την ενσωμάτωση εννοιών από άλλα παραδείγματα **μηχανικής μάθησης**. Για παράδειγμα, η **συνδυαστική εξέλιξη γενετικών κανόνων** με πολιτικές **ενίσχυσης μάθησης**, δομές κανόνων ασαφούς λογικής ή αναπαραστάσεις νευρωνικών δικτύων μπορεί να αυξήσει την ευελιξία και την απόδοση. Τα **υβριδικά συστήματα** θα μπορούσαν να αντιμετωπίσουν προβλήματα που απαιτούν τόσο συμβολική σαφήνεια όσο και συνεχή προσέγγιση λειτουργίας.

11.7 Εφαρμογή σε πραγματικά δεδομένα του κόσμου.

Ενώ το πρόβλημα του **multiplexer** παρέχει ένα εξαιρετικό ελεγχόμενο σημείο αναφοράς, η εφαρμογή του **συστήματος ταξινόμησης** σε πραγματικά σύνολα δεδομένων όπως η ταξινόμηση αισθητήρων, η ανίχνευση ανωμαλιών ή ιατρικές αποφάσεις θα αποδείκνυε την πρακτική χρησιμότητά του. Τα πραγματικά προβλήματα συνήθως περιέχουν θόρυβο, ατελή δεδομένα και μη δυαδικά χαρακτηριστικά, τα οποία θα προκαλούσαν το σύστημα να προσαρμοστεί και να επεκτείνει τη δύναμη αναπαράστασής του.

11.8 Παράλληλη επεξεργασία και υπολογιστική βελτιστοποίηση.

Καθώς η πολυπλοκότητα των προβλημάτων αυξάνεται, τόσο οι ενημερώσεις του **Bucket Brigade** όσο και οι γενετικές λειτουργίες γίνονται ολοένα και πιο απαιτητικές. Η παράλληλη εκτέλεση των αξιολογήσεων κανόνων, η επιτάχυνση των βημάτων μετάλλαξης/διασταύρωσης ή η αξιοποίηση των δυαδικών πράξεων που βασίζονται σε **GPU** θα μείωναν σημαντικά τον χρόνο εκτέλεσης και θα έκαναν το σύστημα επεκτάσιμο σε πολύ μεγαλύτερες πληθυσμιακές ομάδες και πιο σύνθετες εργασίες.

Συνοπτικά, η τρέχουσα εργασία παρέχει μια ισχυρή βάση πάνω στην οποία μπορούν να αναπτυχθούν πολλές προηγμένες λειτουργίες. Με τη βελτίωση της επεκτασιμότητας, την ενίσχυση των ικανοτήτων συλλογισμού, την προσθήκη διαγνωστικών εργαλείων και την επέκταση του μοντέλου σε νέους τομείς, η μελλοντική έρευνα μπορεί να μετατρέψει αυτό το **σύστημα ταξινόμησης** σε ένα πιο ευέλικτο και ισχυρό πλαίσιο μάθησης με βάση κανόνες. Οι προτεινόμενες επεκτάσεις όχι μόνο διευρύνουν την εφαρμοσιμότητα του συστήματος, αλλά συμβάλλουν και σε μια βαθύτερη επιστημονική κατανόηση της **εξελικτικής και προσαρμοστικής συμβολικής μάθησης**.

Bibliography

Foundational Works on Classifier Systems & Holland's Architecture

1. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
— The foundational text introducing genetic algorithms and adaptive rule-based systems.
2. Holland, J. H., Holyoak, K. J., Nisbett, R. E., & Thagard, P. (1986). *Induction: Processes of Inference, Learning, and Discovery*. MIT Press.
— Describes schema processing, rules, and credit assignment similar to the Bucket Brigade.
3. Holland, J. H. (1992). *Genetic Algorithms*. Scientific American.
— Explains genetic operators and evolutionary adaptation used in your rule evolution loop.

Bucket Brigade Algorithm, Strength Update, and Credit Assignment

4. Booker, L. B., Goldberg, D. E., & Holland, J. H. (1989).
“Classifier Systems and Genetic Algorithms.” *Artificial Intelligence*, 40(1–3), 235–282.
— Standard reference describing credit assignment and the Bucket Brigade mechanism.
5. Holland, J. H. (1986). “Escaping Brittleness: The Possibilities of General-Purpose Learning Algorithms Applied to Parallel Rule-Based Systems.” In *Machine Learning: An Artificial Intelligence Approach* (Vol. 2).

— Describes rule strengths, bidding, and reinforcement like in your CS.

Learning Classifier Systems (LCS) Theory

6. Wilson, S. W. (1995). “Classifier Fitness Based on Accuracy.”

Evolutionary Computation, 3(2), 149–175.

— Introduces XCS, an extension of Holland’s CS, widely cited in modern LCS research.

7. Urbanowicz, R. J., & Moore, J. H. (2009). “Learning Classifier Systems: A Complete Introduction, Review, and Roadmap.”

Journal of Artificial Evolution and Applications.

— A broad overview of LCS techniques, GA interactions, and rule accuracy.

Genetic Algorithms & Operators (Roulette Selection, Crossover, Mutation)

8. Goldberg, D. E. (1989). *Genetic Algorithms in Search,*

Optimization, and Machine Learning. Addison-Wesley.

— The classic GA reference, including roulette-wheel selection and multipoint crossover used in your code.

9. Mitchell, M. (1996). *An Introduction to Genetic Algorithms*. MIT Press.

— Very accessible overview of GA mechanisms similar to your implementation.

10.Koza, J. R. (1992). *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press.

— While GP-focused, contains evolutionary reasoning frameworks relevant to your rule evolution.

Multiplexer Benchmark Problem (MUX)

11.Furnkranz, J. (1999). “Separate-and-Conquer Rule Learning.” *Artificial Intelligence Review*, 13(1), 3–54.

— Discusses multiplexer benchmarks and generalization of rule-learning algorithms.

12.De Jong, K. A. (1975). *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. PhD Dissertation, University of Michigan.

— One of the earliest GA works where the multiplexer function appears as a canonical test.

Rule-Based Reasoning and Symbolic Learning

13.Michalski, R. S. (1983). “A Theory and Methodology of Inductive Learning.” *Artificial Intelligence*, 20(2), 111–161.

— A foundational work on symbolic rule induction, relevant to your rule conditions/actions.

14.Quinlan, J. R. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann.

— Though decision-tree based, cited in rule-learning projects for comparison with evolutionary rule approaches.

Software Engineering, Implementation, and Experimental Methodology

15.Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press.

— Standard reference for random number generation and experimental reproducibility.

16.Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*. Addison-Wesley.

— Strong reference for pseudo-randomness theory supporting your RNG-based GA operations.

Optional but useful additions (depending on your thesis depth)

Modern Learning Classifier Systems

17.Butz, M. V. (2006). *Rule-Based Evolutionary Online Learning Systems*. Springer.

— Covers advanced evolutionary and reinforcement mechanisms relevant to your system.

Hybrid Evolutionary Rule Systems

18.Bonarini, A. (1993). “Fuzzy Classifier Systems: A New Reasoning Model.” In *Fuzzy Logic and Soft Computing*.

— Related to future expansion proposals.

Παράρτημα Α

Κώδικας Συστήματος Ταξινόμησης (Classifier System με GA και Bucket Brigade)

Σε αυτό το παράρτημα παρατίθεται ο πλήρης κώδικας σε C++ του συστήματος ταξινόμησης που αναπτύχθηκε στο πλαίσιο της εργασίας. Το σύστημα υλοποιεί:

- Σταθερό πλήθος κανόνων (**Rule**), με πεδία:
 - **condition** (δυαδικό string με χαρακτήρες 0, 1, #)
 - **action** (δυαδική έξοδος)
 - **strength** (ισχύς/πίστωση κανόνα)
- Μηχανισμό **Bucket Brigade** για ενίσχυση/αποδυνάμωση κανόνων ανάλογα με τη σωστή ή λανθασμένη ταξινόμηση.
- **Γενετικό Αλγόριθμο** (γεννήτορες, επιλογή ρουλέτας, πολυσημειακή διασταύρωση, μετάλλαξη, ελιτισμός) για εξέλιξη του συνόλου κανόνων.
- Εκπαίδευση πάνω στο πρόβλημα **4-to-1 multiplexer** (6-bit είσοδος, 1-bit έξοδος).
- Πολλαπλά runs, υπολογισμό ακρίβειας και καταγραφή αποτελεσμάτων σε αρχείο κειμένου.

Ακολουθεί ο πλήρης κώδικας:

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <random>
5  #include <map>
6  #include <iterator>
7  #include <algorithm>
8  #include <numeric>
9  #include <ctime>
10 #include <chrono>
11 #include <iomanip>
12 #include <sstream>
13 #include <fstream>
14
15 struct Rule {
16     std::string condition;
17     std::string action;
18     double strength;
19     Rule(const std::string& condition, const std::string& action, double strength)
20         : condition(condition), action(action), strength(strength) {}
21 };
22
23 // Global random engine
24 std::mt19937 rng(std::random_device{}());
25
26 // Function using the global RNG
27 double rnd() {
28     std::uniform_real_distribution<double> dist(0.0, 1.0);
29     return dist(rng);
30 }
31
32 char mutateChar(char currentChar, std::mt19937& rng) {
33     static const std::map<char, std::vector<char>> alternatives = {
34         {'0', {'1', '#'}},
35         {'1', {'0', '#'}},
36         {'#', {'0', '1'}}
37     };
38     std::uniform_int_distribution<int> dist(0, 1);
39     auto it = alternatives.find(currentChar);
40     if (it != alternatives.end()) {
41         return it->second[dist(rng)];
42     }
43     return currentChar;
44 }
45
46 Rule generateRandomRule(int conditionLength, int actionLength) {
47     std::string condition;
48     std::string action;
49     for (int i = 0; i < conditionLength; ++i) {
50         condition += "01#" [rng() % 3];
51     }
52     for (int i = 0; i < actionLength; ++i) {
53         action += "01" [rng() % 2];
54     }
55     double initialStrength = 10.0;
56     return Rule(condition, action, initialStrength);
57 }
58
59 void mutate(Rule& rule, double mutationRate, std::mt19937& rng) {
60     //std::cout << "Before mutation: Condition=" << rule.condition << ", Action=" << rule.action << std::endl;
61     for (auto& c : rule.condition) {
62         if (rnd() < mutationRate) {
63             //char oldChar = c;
64             c = mutateChar(c, rng);
65             //std::cout << "Mutated '" << oldChar << "' to '" << c << "' in condition." << std::endl;
66         }
67     }
68     for (auto& c : rule.action) {
69         if (rnd() < mutationRate) {
70             //char oldChar = c;
71             c = (c == '0') ? '1' : '0';
72             //std::cout << "Mutated '" << oldChar << "' to '" << c << "' in action." << std::endl;
73         }
74     }
75     //std::cout << "After mutation: Condition=" << rule.condition << ", Action=" << rule.action << std::endl;
76 }

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

77  Rule rouletteWheelSelection(const std::vector<Rule>& rules) {
78      double totalStrength = std::accumulate(rules.begin(), rules.end(), 0.0,
79          [](double sum, const Rule& r) { return sum + r.strength; });
80      double value = rnd() * totalStrength;
81      double partialSum = 0.0;
82      for (const auto& rule : rules) {
83          partialSum += rule.strength;
84          if (partialSum > value || &rule == &rules.back()) { // Ensure the last rule can be selected
85              return rule;
86          }
87      }
88      return rules.back(); // Fallback
89  }

92  void multiPointCrossover(Rule& parent1, Rule& parent2) {
93      // Combine condition and action for crossover purposes
94      std::string genotype1 = parent1.condition + parent1.action;
95      std::string genotype2 = parent2.condition + parent2.action;
96
97      int point1 = rand() % genotype1.size();
98      int point2 = rand() % genotype1.size();
99      if (point1 > point2) {
100          std::swap(point1, point2); // Ensure point1 is less than point2
101      }
102      //std::cout << "Before crossover:\n";
103      //std::cout << "Parent1: " << genotype1 << "\nParent2: " << genotype2 << std::endl;
104      //std::cout << "Crossover points: " << point1 << ", " << point2 << std::endl;
105
106      // Perform crossover between point1 and point2
107      for (int i = point1; i <= point2; ++i) {
108          std::swap(genotype1[i], genotype2[i]);
109      }
110      //std::cout << "After crossover:\n";
111      //std::cout << "Parent1: " << genotype1 << "\nParent2: " << genotype2 << std::endl;
112
113      // Split the genotypes back into condition and action
114      parent1.condition = genotype1.substr(0, parent1.condition.size());
115      parent1.action = genotype1.substr(parent1.condition.size());
116      parent2.condition = genotype2.substr(0, parent2.condition.size());
117      parent2.action = genotype2.substr(parent2.condition.size());
118
119  }

122  bool check_match(const std::string& input, const Rule& rule) {
123      for (size_t i = 0; i < input.size(); ++i) {
124          if (rule.condition[i] != '#' && rule.condition[i] != input[i]) {
125              return false;
126          }
127      }
128      return true;
129  }

130  // Function to find elite rules without sorting the entire population
131  std::vector<int> findEliteIndices(const std::vector<Rule>& rules, int elitismCount) {
132      std::vector<int> eliteIndices(elitismCount, -1);
133      std::vector<double> eliteStrengths(elitismCount, std::numeric_limits<double>::lowest());
134
135      for (int i = 0; i < rules.size(); ++i) {
136          for (int j = 0; j < elitismCount; ++j) {
137              if (rules[i].strength > eliteStrengths[j]) {
138                  // Shift the lower elements down
139                  for (int k = elitismCount - 1; k > j; --k) {
140                      eliteStrengths[k] = eliteStrengths[k - 1];
141                      eliteIndices[k] = eliteIndices[k - 1];
142                  }
143                  // Place the new elite
144                  eliteStrengths[j] = rules[i].strength;
145                  eliteIndices[j] = i;
146                  break;
147              }
148          }
149      }
150      return eliteIndices;
151  }

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

152 void geneticAlgorithm(std::vector<Rule>& rules, double mutationRate, int generation, const std::vector<std::pair<std::string,
153   std::string>>& trainingSet, double elitismRate) {
154   int elitismCount = static_cast<int>(rules.size() * elitismRate); // For example, 10% elitism
155   std::vector<int> eliteIndices = findEliteIndices(rules, elitismCount);
156   std::vector<Rule> newGeneration;
157   newGeneration.reserve(rules.size()); // Reserve space to avoid reallocations
158   // Copy elite rules to the new generation
159   for (int index : eliteIndices) {
160     if (index >= 0 && index < rules.size()) { // Additional bounds checking
161       newGeneration.push_back(rules[index]);
162     }
163   }
164   // Select, crossover, and mutate to create new offspring
165   while (newGeneration.size() < rules.size()) {
166     Rule parent1 = rouletteWheelSelection(rules);
167     Rule parent2 = rouletteWheelSelection(rules);
168     multiPointCrossover(parent1, parent2);
169
170     // Calculate average strength of parents
171     double averageStrength = (parent1.strength + parent2.strength) / 2.0;
172
173     // Assign average strength to both offspring before mutation
174     parent1.strength = averageStrength;
175     parent2.strength = averageStrength;
176
177     // Print initial strengths after crossover but before mutation
178     //std::cout << "Initial Strengths (Before Mutation) - Parent1: " << parent1.strength << ", Parent2: " << parent2.strength << std::endl;
179
180     // Mutate and re-evaluate fitness based on new genotype
181     mutate(parent1, mutationRate, rng);
182     mutate(parent2, mutationRate, rng);
183     // Print strengths after mutation
184     //std::cout << "Strengths After Mutation - Parent1: " << parent1.strength << ", Parent2: " << parent2.strength << std::endl;
185
186     // Add offspring to the new generation
187     newGeneration.push_back(parent1);
188     if (newGeneration.size() < rules.size()) {
189       newGeneration.push_back(parent2);
190     }
191   }
192   // Replace old generation with new generation
193   rules.swap(newGeneration);
194 }
195
196 void shuffleTrainingSet(std::vector<std::pair<std::string, std::string>>& trainingSet) {
197   static std::mt19937 rng(std::random_device{}());
198   std::shuffle(trainingSet.begin(), trainingSet.end(), rng);
199 }
200 Rule generateRuleForSample(const std::string& input, double initialStrength, int actionLength) {
201   std::string condition;
202   std::uniform_int_distribution<int> dist(0, 2); // For '0', '1', or '#'
203   for (char c : input) {
204     int decision = dist(rng);
205     condition += (decision < 2) ? c : '#';
206   }
207
208   std::string action;
209   std::uniform_int_distribution<int> actionDist(0, 1);
210   for (int i = 0; i < actionLength; ++i) {
211     action += (actionDist(rng) == 0) ? '0' : '1';
212   }
213
214   return Rule(condition, action, initialStrength);
215 }
216
217 // Function to find the index of the least effective rule
218 int findIndexofLeastEffectiveRule(const std::vector<Rule>& rules) {
219   double minStrength = std::numeric_limits<double>::max();
220   int index = -1;
221   for (int i = 0; i < rules.size(); ++i) {
222     if (rules[i].strength < minStrength) {
223       minStrength = rules[i].strength;
224       index = i;
225     }
226   }
227   return index;
228 }
229

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

230 bool runBucketBrigade(std::vector<Rule>& rules, std::vector<std::pair<std::string, std::string>>& trainingSet,
231 double bidPercentage, double rewardAmount, double negativeRewardFactor, int repeatCount, double taxRate, int maxRules) {
232 int correctPredictions = 0;
233
234 while (repeatCount-- > 0) {
235 shuffleTrainingSet(trainingSet);
236 for (const auto& ts : trainingSet) {
237 const std::string& input = ts.first;
238 const std::string& expectedOutput = ts.second;
239
240 std::vector<Rule*> matchedRules;
241 for (auto& rule : rules) {
242 if (check_match(input, rule)) {
243 matchedRules.push_back(&rule);
244 }
245 }
246
247 Rule* bestRule = nullptr;
248 if (!matchedRules.empty()) {
249 // Sort matchedRules by strength in descending order
250 std::sort(matchedRules.begin(), matchedRules.end(), [](const Rule* a, const Rule* b) {
251 return a->strength > b->strength;
252 });
253
254 // Handle ties for the top position
255 std::vector<Rule*> topTiedRules;
256 double topStrength = matchedRules.front()->strength;
257 std::copy_if(matchedRules.begin(), matchedRules.end(), std::back_inserter(topTiedRules), [topStrength](const Rule* rule) {
258 return rule->strength == topStrength;
259 });
260
261 if (topTiedRules.size() > 1) {
262 // If there's a tie, randomly select one
263 std::uniform_int_distribution<size_t> dist(0, topTiedRules.size() - 1);
264 bestRule = topTiedRules[dist(rng)];
265 }
266 else {
267 bestRule = matchedRules.front();
268 }
269
270 if (bestRule) {
271 double bidAmount = bestRule->strength * bidPercentage;
272 bestRule->strength -= bidAmount;
273
274 if (bestRule) {
275 double bidAmount = bestRule->strength * bidPercentage;
276 bestRule->strength -= bidAmount;
277
278 if (bestRule->action == expectedOutput) {
279 bestRule->strength += rewardAmount;
280 correctPredictions++;
281 }
282 else {
283 // Apply negative reward if the prediction is incorrect
284 double negativeReward = bidAmount * negativeRewardFactor; // Calculate negative reward
285 bestRule->strength -= negativeReward; // Deduct negative reward
286 if (bestRule->strength < 0) bestRule->strength = 0; // Ensure strength doesn't go below 0
287 }
288 }
289 else if (rules.size() >= maxRules) {
290 int leastEffectiveRuleIndex = findIndexOfLeastEffectiveRule(rules);
291 if (leastEffectiveRuleIndex != -1) {
292 // Replace the least effective rule with a new rule based on the unmatched input
293 // generateRuleForSample only takes two parameters.
294 rules[leastEffectiveRuleIndex] = generateRuleForSample(input, 10.0, 1);
295 }
296 }
297 else {
298 // Directly add a new rule if under the max limit
299 rules.push_back(generateRuleForSample(input, 10.0, 1));
300 }
301
302 // Apply a tax to all rules
303 for (auto& rule : rules) {
304 rule.strength *= (1.0 - taxRate);
305 }
306
307 return true;
308 }
309
310 // Function to predict the output for a given input based on the rules, using strength and tie resolution
311 std::string predictOutput(const std::vector<Rule>& rules, const std::string& input) {
312 std::vector<const Rule*> matchedRules;
313

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

314 // Collect all rules that match the input
315 for (const auto& rule : rules) {
316     if (std::equal(rule.condition.begin(), rule.condition.end(), input.begin(),
317         [](char r, char i) { return r == '#' || r == i; })) {
318         matchedRules.push_back(rule);
319     }
320 }
321
322 if (matchedRules.empty()) {
323     return ""; // Return an empty string if no rule matches
324 }
325
326 // Sort matched rules by strength in descending order
327 std::sort(matchedRules.begin(), matchedRules.end(), [](const Rule* a, const Rule* b) {
328     return a->strength > b->strength;
329 });
330
331 // Handle ties by selecting a rule randomly if there's a tie at the top
332 std::vector<const Rule*> topTiedRules;
333 double topStrength = matchedRules.front()->strength;
334 std::copy_if(matchedRules.begin(), matchedRules.end(), std::back_inserter(topTiedRules), [topStrength](const Rule* rule) {
335     return rule->strength == topStrength;
336 });
337
338 static std::mt19937 rng(std::random_device{}()); // Use a static rng to avoid frequent re-seeding
339 if (topTiedRules.size() > 1) {
340     std::uniform_int_distribution<size_t> dist(0, topTiedRules.size() - 1);
341     return topTiedRules[dist(rng)]->action;
342 }
343
344 return matchedRules.front()->action; // Return the action of the rule with the highest strength
345 }
346
347 int main() {
348     srand(static_cast<unsigned>(time(0)));
349
350     int populationSize = 45, maxRules = 60, maxGenerations = 2000;
351     double mutationRate = 0.01;
352     double bidPercentage = 0.05;
353     double rewardAmount = 2.0; // This Parameter gives a Reward to the Rules if they are correct .
354     int repeatCount = 20;
355     double taxRate = 0.10;
356
357     double negativeRewardFactor = 1.25; // This Parameter gives a negative Reward to the Rules if they are not correct .
358
359     double strengthThreshold = 25; //This is the value that choose the
360     double elitismRate = 0.25; // To take 10% of the rules size we need to give 0.1 !!
361
362     int conditionLength = 6;
363     int actionLength = 1;
364     // Set precision for floating-point numbers
365     std::cout << std::fixed << std::setprecision(2);
366
367     // Print the table header
368     std::cout << std::setw(30) << "Parameter" << std::setw(15) << "Value" << std::endl;
369     std::cout << std::string(45, '-') << std::endl;
370
371     // Print the parameters in a tabular format
372     std::cout << std::setw(30) << "Population Size" << std::setw(15) << populationSize << std::endl;
373     std::cout << std::setw(30) << "Max Rules" << std::setw(15) << maxRules << std::endl;
374     std::cout << std::setw(30) << "Max Generations" << std::setw(15) << maxGenerations << std::endl;
375     std::cout << std::setw(30) << "Mutation Rate" << std::setw(15) << mutationRate << std::endl;
376     std::cout << std::setw(30) << "Bid Percentage" << std::setw(15) << bidPercentage << std::endl;
377     std::cout << std::setw(30) << "Reward Amount" << std::setw(15) << rewardAmount << std::endl;
378     std::cout << std::setw(30) << "Repeat Count" << std::setw(15) << repeatCount << std::endl;
379     std::cout << std::setw(30) << "Tax Rate" << std::setw(15) << taxRate << std::endl;
380     std::cout << std::setw(30) << "Negative Reward Factor" << std::setw(15) << negativeRewardFactor << std::endl;
381     std::cout << std::setw(30) << "Strength Threshold" << std::setw(15) << strengthThreshold << std::endl;
382     std::cout << std::setw(30) << "Elitism Rate" << std::setw(15) << elitismRate << std::endl;
383     std::cout << std::setw(30) << "Condition Length" << std::setw(15) << conditionLength << std::endl;
384     std::cout << std::setw(30) << "Action Length" << std::setw(15) << actionLength << std::endl;
385
386     std::cout << std::string(45, '-') << std::endl;
387

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

388     std::vector<std::pair<std::string, std::string>> trainingSet = {
389         {"000000", "0"}, {"000001", "0"}, {"000010", "0"}, {"000011", "0"},
390         {"000100", "0"}, {"000101", "0"}, {"000110", "0"}, {"000111", "0"},
391         {"001000", "1"}, {"001001", "1"}, {"001010", "1"}, {"001011", "1"},
392         {"001100", "1"}, {"001101", "1"}, {"001110", "1"}, {"001111", "1"},
393
394         {"010000", "0"}, {"010001", "0"}, {"010010", "0"}, {"010011", "0"},
395         {"010100", "1"}, {"010101", "1"}, {"010110", "1"}, {"010111", "1"},
396         {"011000", "0"}, {"011001", "0"}, {"011010", "0"}, {"011011", "0"},
397         {"011100", "1"}, {"011101", "1"}, {"011110", "1"}, {"011111", "1"},
398
399         {"100000", "0"}, {"100001", "0"}, {"100010", "1"}, {"100011", "1"},
400         {"100100", "0"}, {"100101", "0"}, {"100110", "1"}, {"100111", "1"},
401         {"101000", "0"}, {"101001", "0"}, {"101010", "1"}, {"101011", "1"},
402         {"101100", "0"}, {"101101", "0"}, {"101110", "1"}, {"101111", "1"},
403
404         {"110000", "0"}, {"110001", "1"}, {"110010", "0"}, {"110011", "1"},
405         {"110100", "0"}, {"110101", "1"}, {"110110", "0"}, {"110111", "1"},
406         {"111000", "0"}, {"111001", "1"}, {"111010", "0"}, {"111011", "1"},
407         {"111100", "0"}, {"111101", "1"}, {"111110", "0"}, {"111111", "1"},
408     };
409
410     // Initialize the rules.
411     std::vector<Rule> rules;
412     rules.reserve(maxRules); // Reserve space to optimize memory allocations
413     for (int i = 0; i < populationSize; ++i) {
414         rules.push_back(generateRandomRule(conditionLength, actionLength));
415     }
416
417     // Prepare to capture accuracies
418     std::vector<double> accuracies(10); // To store accuracies from each run
419
420     // Get current time safely with localtime_s on MSVC
421     std::time_t now = std::time(nullptr);
422     std::tm timeStruct = {}; // initialize to zero
423
424     char buffer[80];
425     localtime_s(&timeStruct, &now); // Safely copy time to timeStruct
426     std::strftime(buffer, sizeof(buffer), "%Y-%m-%d_%H-%M-%S", &timeStruct); // Format the time
427     std::string filename = "final_results_" + std::string(buffer) + ".txt";
428
429     // Open text file to write final results
430     // Make sure to open the file only once
431     std::ofstream outFile(filename);
432     if (!outFile.is_open()) {
433         std::cerr << "Failed to open file: " << filename << "\n";
434         return 1;
435     }
436
437     // Set precision for floating-point numbers
438     std::cout << std::fixed << std::setprecision(2);
439
440     // Print the table header
441     outFile << std::setw(30) << "Parameter" << std::setw(15) << "Value" << std::endl;
442     outFile << std::string(45, '-') << std::endl;
443
444     // Print the parameters in a tabular format
445     outFile << std::setw(30) << "Population Size" << std::setw(15) << populationSize << std::endl;
446     outFile << std::setw(30) << "Max Rules" << std::setw(15) << maxRules << std::endl;
447     outFile << std::setw(30) << "Max Generations" << std::setw(15) << maxGenerations << std::endl;
448     outFile << std::setw(30) << "Mutation Rate" << std::setw(15) << mutationRate << std::endl;
449     outFile << std::setw(30) << "Bid Percentage" << std::setw(15) << bidPercentage << std::endl;
450     outFile << std::setw(30) << "Reward Amount" << std::setw(15) << rewardAmount << std::endl;
451     outFile << std::setw(30) << "Repeat Count" << std::setw(15) << repeatCount << std::endl;
452     outFile << std::setw(30) << "Tax Rate" << std::setw(15) << taxRate << std::endl;
453     outFile << std::setw(30) << "Negative Reward Factor" << std::setw(15) << negativeRewardFactor << std::endl;
454     outFile << std::setw(30) << "Strength Threshold" << std::setw(15) << strengthThreshold << std::endl;
455     outFile << std::setw(30) << "Elitism Rate" << std::setw(15) << elitismRate << std::endl;
456     outFile << std::setw(30) << "Condition Length" << std::setw(15) << conditionLength << std::endl;
457     outFile << std::setw(30) << "Action Length" << std::setw(15) << actionLength << std::endl;
458
459     outFile << std::string(45, '-') << std::endl;
460
461
462

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

463     for (int runcounter = 0; runcounter < 10; ++runcounter)
464     {
465         std::cout << "Run : " << runcounter << "\n" << std::endl;
466         outFile << "Run : " << runcounter << "\n" << std::endl;
467
468         for (int gen = 0; gen < maxGenerations; ++gen) {
469
470             runBucketBrigade(rules, trainingSet, bidPercentage, rewardAmount, negativeRewardFactor, repeatCount, taxRate, maxRules);
471             geneticAlgorithm(rules, mutationRate, gen, trainingSet, elitismRate);
472         }
473
474         std::vector<Rule> highStrengthRules;
475
476         std::copy_if(rules.begin(), rules.end(), std::back_inserter(highStrengthRules), [strengthThreshold](const Rule& rule) {
477             return rule.strength > strengthThreshold;
478         });
479
480         int correctPredictions = 0;
481         for (const auto& ts : trainingSet) {
482             if (predictOutput(rules, ts.first) == ts.second) {
483                 correctPredictions++;
484             }
485         }
486
487         double accuracy = static_cast<double>(correctPredictions) / trainingSet.size() * 100.0;
488         accuracies[runcounter] = accuracy; // Store accuracy for this run
489
490
491         outFile << "Final High Strength Rules:\n";
492         std::cout << "Final High Strength Rules:\n";
493         for (const auto& rule : highStrengthRules) {
494             outFile << "Condition: " << rule.condition << " Action: " << rule.action << " Strength: " << rule.strength << "\n" << std::endl;
495             std::cout << "Condition: " << rule.condition << " Action: " << rule.action << " Strength: " << rule.strength << "\n";
496         }
497
498
499         // Write the final rules
500
501         outFile << "ALL RULES " << "\n" << std::endl;
502         for (int i = 0; i < rules.size(); ++i) {
503             outFile << "Rule " << i + 1 << ": Condition = " << rules[i].condition
504                 << ", Action = " << rules[i].action
505                 << ", Strength = " << rules[i].strength << "\n" << std::endl;
506         }
507
508         std::cout << "Final Accuracy: " << accuracy << "%\n";
509         // Write the accuracy
510         outFile << "\nFinal Accuracy: " << accuracy << "%\n" << std::endl;
511
512         std::cout << "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX\n";
513     }
514
515     // Compute average and maximum accuracies
516     double averageAccuracy = std::accumulate(accuracies.begin(), accuracies.end(), 0.0) / accuracies.size();
517     double maxAccuracy = *std::max_element(accuracies.begin(), accuracies.end());
518     outFile << "
519         \n";
520     outFile << "
521         \n";
522     outFile << "Average Accuracy: " << averageAccuracy << "%\n" << std::endl;
523     outFile << "Maximum Accuracy: " << maxAccuracy << "%\n" << std::endl;
524
525     outFile.close();
526     if (outFile.fail()) {
527         std::cerr << "Error occurred during file operation on: " << filename << "\n";
528         return 1;
529     }
530
531     return 0;
532 }
533
534

```

Παράρτημα Β

Κώδικας Δεύτερης Εφαρμογής Κανόνων για Εξαγωγή Συμπερασμάτων

Σε αυτό το παράρτημα παρουσιάζεται ο δεύτερος κώδικας C++ που αναπτύχθηκε στο πλαίσιο της εργασίας. Η εφαρμογή αυτή **δεν εκπαιδεύει** το σύστημα ταξινόμησης, αλλά χρησιμοποιεί τους κανόνες που έχουν ήδη παραχθεί από τον **Γενετικό Αλγόριθμο (GA)** και καταγραφεί στο αρχείο αποτελεσμάτων (ενότητα **ALL RULES**), με σκοπό:

- να τους **φορτώσει** από επιλεγμένο αρχείο κειμένου,
- να τους **εφαρμόσει** σε όλες τις 64 δυνατές 6-bit εισόδους του πολυπλέκτη,
- να υπολογίσει για κάθε κανόνα πόσες φορές:
 - ταιριάζει με την είσοδο,
 - δίνει **σωστή ή λανθασμένη** έξοδο,
- να εμφανίσει για κάθε είσοδο ποιους κανόνες ενεργοποιεί και ποιος «κερδίζει» (κανόνας με τη μεγαλύτερη strength),
- να εκτυπώσει στο τέλος μία συγκεντρωτική **αναφορά απόδοσης κανόνων**, τονίζοντας όσους δεν έκαναν ποτέ λάθος.

Έτσι, ο κώδικας αυτός λειτουργεί ως **εργαλείο ανάλυσης** του τελικού συνόλου κανόνων, επιτρέποντας να εξαχθούν συμπεράσματα για την ποιότητα, τη γενίκευση και την αξιοπιστία των κανόνων που παρήγαγε το σύστημα ταξινόμησης.

Ακολουθεί ο πλήρης κώδικας:

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

1  #include <iostream>
2  #include <fstream>
3  #include <sstream>
4  #include <vector>
5  #include <string>
6  #include <algorithm>
7  #include <windows.h>
8  #include <commdlg.h>
9  #include <regex>
10
11 struct Rule {
12     std::string condition;
13     int action;
14     double strength;
15     int success = 0;
16     int failures = 0;
17 };
18
19 // Check if a rule condition matches a data input
20 bool matches(const std::string& cond, const std::string& data) {
21     if (cond.size() != data.size()) return false;
22     for (size_t i = 0; i < cond.size(); ++i) {
23         if (cond[i] != '#' && cond[i] != data[i])
24             return false;
25     }
26     return true;
27 }
28
29 // Multiplexer function: returns the correct action for a 6-bit input
30 int multiplexer6bit(const std::string& data) {
31     int addr = (data[0] - '0') * 2 + (data[1] - '0'); // first 2 bits as address
32     return data[2 + addr] - '0'; // select the addressed data bit
33 }
34
35 // Open file selection dialog and return selected path
36 std::string selectFile() {
37     OPENFILENAMEW ofn;
38     wchar_t szFile[260] = { 0 };
39
40     ZeroMemory(&ofn, sizeof(ofn));
41     ofn.lStructSize = sizeof(ofn);
42     ofn.lpstrFile = szFile;
43     ofn.nMaxFile = sizeof(szFile) / sizeof(wchar_t);
44     ofn.lpstrFilter = L"Text Files\\0*.txt\\0All Files\\0*.*\\0";
45     ofn.Flags = OFN_PATHMUSTEXIST | OFN_FILEMUSTEXIST | OFN_NOCHANGEDIR;
46     ofn.lpstrTitle = L"Select Rule File";
47
48     if (GetOpenFileNameW(&ofn) == TRUE) {
49         int size_needed = WideCharToMultiByte(CP_UTF8, 0, szFile, -1, nullptr, 0, nullptr, nullptr);
50         std::string filePath(size_needed, 0);
51         WideCharToMultiByte(CP_UTF8, 0, szFile, -1, &filePath[0], size_needed, nullptr, nullptr);
52         if (!filePath.empty() && filePath.back() == '\\0') filePath.pop_back();
53         return filePath;
54     }
55     return "";
56 }
57
58 // Parse a line in "ALL RULES" section
59 bool parseAllRulesLine(const std::string& line, Rule& rule) {
60     // Format: Rule 1: Condition = 11##1, Action = 1, Strength = 30.0725
61     std::regex re(R"(Rule\s*\d+:\s*Condition\s*=\s*(\S+),\s*Action\s*=\s*(\d+),\s*Strength\s*=\s*([0-9]*\.\?[0-9]+))");
62     std::smatch match;
63     if (std::regex_match(line, match, re)) {
64         rule.condition = match[1].str();
65         rule.action = std::stoi(match[2].str());
66         rule.strength = std::stod(match[3].str());
67         return true;
68     }
69     return false;
70 }
71
72 // Set console text color
73 void setConsoleColor(WORD color) {
74     SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
75 }
76

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```

77  int main() {
78      std::string filepath = selectFile();
79      if (filepath.empty()) {
80          std::cerr << "No file selected.\n";
81          return 1;
82      }
83
84      std::ifstream file(filepath);
85      if (!file) {
86          std::cerr << "Error opening file.\n";
87          return 1;
88      }
89
90      std::vector<Rule> rules;
91      std::string line;
92      bool readingRules = false;
93
94      while (std::getline(file, line)) {
95          if (line.find("ALL RULES") != std::string::npos) {
96              readingRules = true;
97              continue;
98          }
99
100         if (readingRules) {
101             Rule r;
102             if (parseAllRulesLine(line, r)) {
103                 rules.push_back(r);
104             }
105         }
106     }
107
108     if (rules.empty()) {
109         std::cerr << "No rules found in ALL RULES section.\n";
110         return 1;
111     }
112
113     // Sort rules by strength descending for initial display
114     std::sort(rules.begin(), rules.end(), [](const Rule& a, const Rule& b) {
115         return a.strength > b.strength;
116     });
117
118     std::cout << "=== Rules (from ALL RULES section, sorted by strength) ===\n";
119     for (size_t i = 0; i < rules.size(); ++i) {
120         std::cout << "Rule " << i + 1
121             << ": Condition = " << rules[i].condition
122             << ", Action = " << rules[i].action
123             << ", Strength = " << rules[i].strength
124             << ", Success = 0, Failures = 0\n";
125     }
126     std::cout << "\n";
127
128     // --- Collect rules per run ---
129     struct Run {
130         std::string data;
131         std::vector<int> matchingRules; // indices of matching rules
132         int winner = -1; // index of winning rule
133     };
134
135     std::vector<Run> runs;
136
137     for (int i = 0; i < 64; ++i) {
138         std::string data;
139         for (int b = 5; b >= 0; --b)
140             data += ((i >> b) & 1) ? '1' : '0';
141
142         int correct_output = multiplexer6bit(data);
143         Run currentRun;
144         currentRun.data = data;
145
146         for (size_t r = 0; r < rules.size(); ++r) {
147             if (matches(rules[r].condition, data)) {
148                 currentRun.matchingRules.push_back(r);
149                 if (rules[r].action == correct_output)
150                     rules[r].success++;
151                 else
152                     rules[r].failures++;
153             }
154         }
155     }

```

Υλοποίηση ενός Classifier System σταθερού πλήθους κανόνων με δυνατότητα αλυσιδωτής ενεργοποίησης κανόνων (συμπερασμού) για εφαρμογή σε προβλήματα κατηγοριοποίησης.

```
156 // Determine winner by highest strength
157 double max_strength = -1.0;
158 for (size_t idx = 0; idx < currentRun.matchingRules.size(); ++idx) {
159     int ruleIdx = currentRun.matchingRules[idx];
160     if (rules[ruleIdx].strength > max_strength) {
161         max_strength = rules[ruleIdx].strength;
162         currentRun.winner = ruleIdx;
163     }
164 }
165
166 runs.push_back(currentRun);
167 }
168
169 // --- Print all runs at the end ---
170 for (size_t runNum = 0; runNum < runs.size(); ++runNum) {
171     const Run& r = runs[runNum];
172     std::cout << "\n=== Run " << (runNum + 1) << " (Data = " << r.data << ") ===\n";
173     if (r.matchingRules.empty()) {
174         std::cout << "No rule activated for this data input.\n";
175         continue;
176     }
177
178     for (size_t i = 0; i < r.matchingRules.size(); ++i) {
179         int ruleIdx = r.matchingRules[i];
180         if ((int)ruleIdx == r.winner)
181             setConsoleColor(10); // green for winner
182
183         std::cout << "Rule " << ruleIdx + 1
184             << ": Condition = " << rules[ruleIdx].condition
185             << ", Action = " << rules[ruleIdx].action
186             << ", Strength = " << rules[ruleIdx].strength
187             << ", Success = " << rules[ruleIdx].success
188             << ", Failures = " << rules[ruleIdx].failures
189             << "\n";
190
191         if ((int)ruleIdx == r.winner)
192             setConsoleColor(7); // reset
193     }
194 }
195
```